

CH 1.1: INTRODUCTION TO DISTRIBUTED DATA PROCESSING

Table of Contents

Disadvantages of Centralized Database (Need for DDBMS).....	1
Distributed Data & Database.....	2
Distributed Database Management System (DDBMS)	2
Date's 12 Rules for DDBMS.....	3
Types of DDBMS.....	4
Components of a DDBMS.....	6
Questions.....	7

Disadvantages of Centralized Database (Need for DDBMS)

1. **Single Point of Failure** – Entire system fails if the central server crashes.
2. **Scalability Issues** – Struggles to handle large-scale data and user growth.
3. **Performance Bottleneck** – High traffic can overload the central server.
4. **Slow Access for Remote Users** – Increases latency for distant users.
5. **Limited Availability** – Downtime affects all users and operations.
6. **High Network Load** – Constant data requests slow down performance.
7. **Security Risks** – A single breach can compromise all data.
8. **Data Locality Issues** – Users far from the central server experience delays.
9. **Maintenance Challenges** – Requires strong infrastructure and backup solutions.
10. **Costly Upgrades** – Expanding storage and processing power is expensive.



Feature	Centralized DBMS	Distributed DBMS
Data Storage	Stored at a single location	Stored across multiple sites
Failure Impact	Single point of failure	Fault-tolerant, multiple sites
Scalability	Limited expansion	Easily scalable
Performance	Slower with high traffic	Faster due to load distribution
Access Speed	Slow for remote users	Faster, data closer to users
Maintenance	Easier to manage	Complex due to multiple sites
Cost	Lower setup cost	Higher infrastructure cost
Data Consistency	Easier to maintain	Difficult due to replication
Security	Centralized control	More challenging across sites
Query Processing	Simple, local execution	Distributed, requires optimization

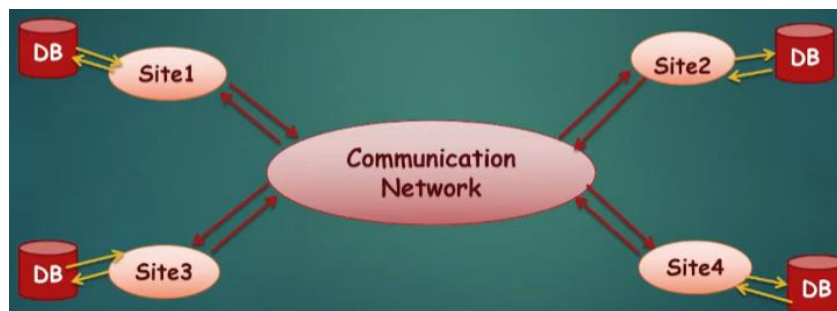
Distributed Data & Database

Distributed Data refers to data that is not stored in a single location but is instead spread across multiple physical or logical locations, such as different servers, cloud storage, or databases. The key idea is that data is stored closer to the point of use, improving accessibility, availability, and performance.

- Stored across multiple sites or servers.
- Managed independently at each site but still accessible globally.
- Can be **replicated** (same data stored in multiple locations) or **fragmented** (different parts of the data stored at different locations).
- Requires a mechanism to ensure **data consistency and synchronization** across all sites.

Distributed database is collection of multiple logically interrelated databases distributed over a computer network (as if they are single database).

- **Database System** – Manages structured data, ensuring storage, retrieval, integrity, and security.
- **Computer Networks** – Enables communication between distributed database nodes across different locations.



Examples of Distributed Data:

- **Banking Systems:** Customer account data is stored at different branch locations but can be accessed globally.
- **Cloud Storage Services:** Google Drive, Dropbox, and OneDrive store user data across multiple data centres.
- **E-commerce Platforms:** Amazon stores inventory data in different warehouses, ensuring fast access to stock information.

Distributed Database Management System (DDBMS)

A **Distributed Database Management System (DDBMS)** is a software system that manages a distributed database, ensuring that the data appears as a single database to users while being stored across multiple locations.

1. **Data Distribution** – Uses fragmentation, replication, and allocation to distribute data.
2. **Query Processing** – Splits queries, optimizes execution, and minimizes overhead.
3. **Transaction Management** – Maintains ACID properties across sites.
4. **Concurrency Control** – Prevents conflicts using locking and timestamps.
5. **Replication & Synchronization** – Ensures consistency across replicated data.
6. **Failure Recovery** – Maintains consistency and system continuity after failures.
7. **Security** – Uses authentication, authorization, and encryption for protection.
8. **Distributed Transactions** – Uses protocols like 2-phase commit to ensure atomicity.

Implicit Assumptions in a Distributed Database System (DDBMS):

1. **Data Distribution** – Data is stored across multiple sites, each functioning as a single logical processor.
2. **Network Connectivity** – Sites communicate via a computer network.
3. **Database Nature** – The system manages structured data, not just a collection of files.
4. **User Access Patterns** – Data placement and query execution depend on user access behavior.
5. **DDBMS Structure** – It is a collection of DBMSs managing distributed data, not a remote file system.

Applications of DDBMS:

1. **Banking**: Manages accounts and transactions across branches.
2. **Telecom**: Handles customer data and call records globally.
3. **E-commerce**: Stores product data and orders across locations.
4. **Cloud Computing**: Distributes user data for scalability.
5. **Healthcare**: Manages patient records across hospitals.
6. **Supply Chain**: Tracks inventory and shipments globally.
7. **Social media**: Stores user data across multiple servers.

Advantages:

1. **Reliability**: Ensures availability even during failures.
2. **Scalability**: Expands easily with more sites.
3. **Locality**: Data stored closer to users.
4. **Performance**: Reduces bottlenecks via distribution.
5. **Control**: Centralized management with distributed access.
6. **Security**: Uses encryption and partitioning.

Disadvantages:

1. **Complexity**: Difficult to design and manage.
2. **Cost**: Expensive to implement and maintain.
3. **Overhead**: High communication and synchronization costs.
4. **Consistency**: Challenging to maintain uniform data.
5. **Security Risks**: Harder to secure multiple sites.

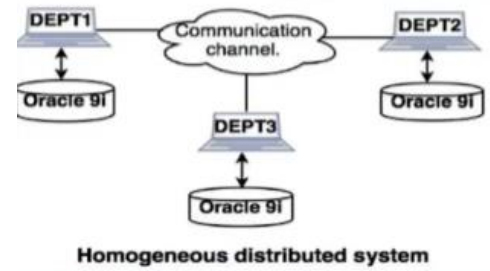
Date's 12 Rules for DDBMS

1. **Local Autonomy** – Each site can function independently.
2. **No Reliance on a Central Site** – No single point of control or failure.
3. **Continuous Operation** – System should remain operational even if some sites fail.
4. **Location Independence** – Users can access data without knowing its physical location.
5. **Fragmentation Independence** – Data can be fragmented across sites without affecting users.
6. **Replication Independence** – Users should not be concerned with whether data is replicated.
7. **Distributed Query Processing** – Queries should be optimized across multiple sites.
8. **Distributed Transaction Management** – ACID properties should be maintained across sites.
9. **Hardware Independence** – Should work across different hardware platforms.
10. **Operating System Independence** – Should function across multiple OS environments.
11. **Network Independence** – Should operate over various network types.
12. **DBMS Independence** – Should support different database systems at different sites.

Types of DDBMS

1. Homogeneous Distributed Database System: All the sites in a homogeneous DDBMS use the same DBMS software and hardware. **Example:** Multiple instances of Oracle DBMS across different locations.

- Consistent data model and schema across all sites.
- Easier to maintain as the software is uniform.
- Simplified data management and administration.
- Seamless integration between different sites.
- Example:** some changes made in one of the 3 departments, will be reflected and updated in other 2.



f. **Types of HDBBMS:**

- Autonomous** – Each site has its own database and transaction management but follows a common schema and DBMS.
- Non-Autonomous** – A central system controls all sites, enforcing uniform transaction management and coordination.

Site Autonomy (Local Autonomy)

Site autonomy (or local autonomy) means that each site in a DDBMS can operate independently without requiring constant coordination from a central authority.

Key Aspects:

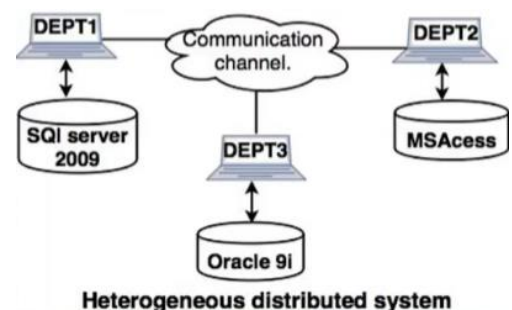
- Independent Control** – Each site manages its own database, transactions, and recovery.
- Local Transaction Execution** – Sites can process local queries without external dependency.
- Failure Isolation** – If one site fails, others continue functioning.
- Decentralized Administration** – No single site has complete control over the entire system.
- Flexibility** – Sites can adopt their own security and access policies.

Types:

- Full Autonomy:** Sites operate entirely independently, with minimal synchronization.
- Partial Autonomy:** Some degree of central coordination exists, but sites have local control over data.

2. Heterogeneous Distributed Database System: The sites in this system may use different DBMS software or hardware, which may lead to variations in data models and schemas. **Example:** Integration of various DBMS like MySQL and MongoDB at different sites.

- Different types of DBMS are used at different sites (e.g., Oracle, MySQL, SQL Server).
- Requires middleware to enable interoperability between different systems.
- Flexibility in selecting the best DBMS for each site.
- Ability to integrate different types of systems.

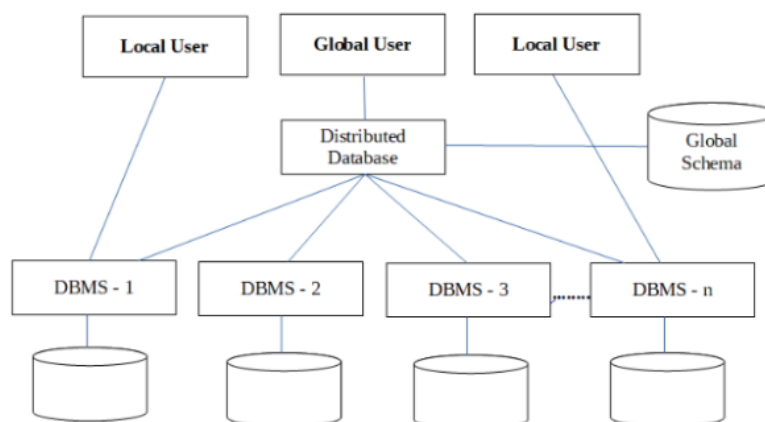


Feature	Homogeneous DDBMS	Heterogeneous DDBMS
Definition	All sites use the same DBMS	Sites use different DBMSs
Schema	Common global schema	Different schemas across sites
Interoperability	High, easy communication	Low, requires translation
Autonomy	Usually non-autonomous	Can be autonomous
Complexity	Lower, easier to manage	Higher, needs middleware
Query Processing	Simple and uniform	Complex, requires integration
Transaction Management	Easier to maintain ACID	More challenging to ensure ACID
Examples	Multiple Oracle DBs across sites	Oracle at one site, MySQL at another

3. **Fragmented Distributed Database System:** The database is divided into fragments (subsets of data), which can be distributed across different sites. **Example:** Customer database fragmented by region, where each regional office stores its own data.
 - a. Data is divided based on rows (horizontal fragmentation), columns (vertical fragmentation), or a combination of both.
 - b. Each site holds only a part of the data, and the database is recreated by collecting fragments.
 - c. Efficient data access and reduced load at each site.
 - d. Localized queries to only relevant data.
4. **Replicated Distributed Database System:** Copies (replicas) of the same data are stored at multiple sites, ensuring data availability and fault tolerance. **Example:** Online services like Amazon replicate their product catalog across multiple servers.
 - a. Data replication improves the reliability and availability of data.
 - b. Updates to data need to be synchronized across all replicas.
 - c. High availability of data even in the event of site failure.
 - d. Faster read access to data due to replication.
5. **Hybrid Distributed Database System:** Combines both fragmented and replicated distributed database systems. Some data is fragmented and distributed, while other data is replicated across sites. **Example:** A global e-commerce platform using fragmented product catalogs and replicated customer data.
 - a. Flexibility in data management, allowing both distribution and replication.
 - b. Some parts of the database are fragmented based on the workload, while others are replicated for redundancy.
 - c. Optimized performance by balancing between fragmentation and replication.
 - d. Scalable and fault-tolerant system.

Components of a DDBMS

A **DDBMS** consists of several components that work together to provide a unified view of data spread across different locations. These components ensure that data is accessible, consistent, and manageable in a distributed environment.



1. **Database Sites:** These are the physical locations where the database is stored. Each site may contain a fragment of the data, or the complete database, depending on the type of distribution (fragmentation or replication). **Types:**
 - **Master Site:** Primary location responsible for central coordination, may hold the complete database.
 - **Slave Sites:** Secondary locations that may store fragments or replicas.
2. **Fragmentation, Replication, and Allocation Control:** This component handles the distribution of data across various sites. It decides how data is fragmented (row/column), replicated, and allocated across different nodes.
3. **Query Processor:** The query processor breaks down user queries and distributes them to appropriate sites for execution. It optimizes query execution by considering where data is stored. **Components:**
 - **Parser:** Checks the syntax and semantics of the query.
 - **Optimizer:** Determines the most efficient execution plan for queries.
 - **Execution Engine:** Executes the query on the relevant site(s).
4. **Transaction Management:** Responsible for ensuring that distributed transactions maintain **ACID (Atomicity, Consistency, Isolation, Durability)** properties across all sites. It manages coordination and synchronization between various sites during a transaction.
5. **Concurrency Control:** Manages simultaneous data access to ensure consistency when multiple users are accessing and modifying data at different sites. **Techniques:** Locking protocols, timestamp ordering, and multi-version concurrency control (MVCC).
6. **Communication Subsystem:** Facilitates communication between distributed database sites. This includes both message passing and data exchange. It ensures reliable data transmission, handles failures, and manages network protocols.

7. **Recovery Subsystem:** Ensures that the database can recover from failures, including network, hardware, and software failures. It guarantees that data remains consistent and the system can resume operations after a failure.
8. **Directory Service:** Keeps track of the metadata, which includes information about where each piece of data is located (i.e., the location of fragments or replicas). It helps in locating data efficiently and guiding the system in accessing relevant data.

Questions

1. Storing a separate copy of the database at multiple locations is which of the following?

- a. Data Replication
- b. Horizontal Partitioning
- c. Vertical Partitioning
- d. None of the above

✓ **Answer:** a. Data Replication

2. A distributed database is a collection of data which belong _____ to the same system but are spread over the _____ of the network.

- a. Logically, sites
- b. Physically, sites
- c. Database, DBMS
- d. None of the above

✓ **Answer:** a. Logically, sites

3. Which of the following is/are the main goals of a distributed database?

- a. Interconnection of database
- b. Incremental growth
- c. Reduced communication overhead
- d. All of the above

✓ **Answer:** d. All of the above

4. Which of the following is a distributed database?

- a. A single logical database that is spread to multiple locations and is interconnected by a network
- b. A loose collection of a file that is spread to multiple locations and is interconnected by a network
- c. A single logical database that is limited to one location
- d. None

✓ **Answer:** a. A single logical database that is spread to multiple locations and is interconnected by a network

5. Which of the following strategies is used by a distributed database?

- a. Totally centralized at one location and accessed by many sites
- b. Totally or partially at one location and distributed at many sites
- c. Partitioned into segments at different sites
- d. All of the above

✓ **Answer:** d. All of the above

6. An autonomous homogeneous environment is which of the following?

- a. The same DBMS is at each node and each DBMS works independently.
- b. The same DBMS is at each node and a central DBMS coordinates database access.
- c. A different DBMS is at each node and each DBMS works independently.
- d. A different DBMS is at each node and a central DBMS coordinates database access.

☒ **Answer:** a. The same DBMS is at each node and each DBMS works independently.

7. A heterogeneous distributed database is which of the following?

- a. The same DBMS is used at each location and data are not distributed across all nodes.
- b. The same DBMS is used at each location and data are distributed across all nodes.
- c. A different DBMS is used at each location and data are not distributed across all nodes.
- d. A different DBMS is used at each location and data are distributed across all nodes.

☒ **Answer:** d. A different DBMS is used at each location and data are distributed across all nodes.

8. Which of the following is not one of the stages in the evolution of distributed DBMS?

- a. Unit of work
- b. Remote unit of work
- c. Distributed unit of work
- d. Distributed request

☒ **Answer:** a. Unit of work

9. Databases that are stored on computers at multiple locations and are not interconnected by a network are known as distributed databases.

- a. True
- b. False

☒ **Answer:** b. False

10. Which of the following is the advantage of a distributed database over a centralized database?

- a. Module wise growth
- b. Simplicity and ease
- c. Slow Response
- d. None of the above

☒ **Answer:** a. Module wise growth

11. Which DDBMS provides better _____ over centralized DBMS?

- a. Decentralization
- b. Tuning
- c. Security
- d. Transparency

☒ **Answer:** a. Decentralization

12. Which of the following features of DDBMS?

- a. Distribution
- b. Logical correlation
- c. Distribution & logical correlation
- d. Disjointness

☒ **Answer:** c. Distribution & logical correlation

13. What is site autonomy?

- a. Each server participating in a distributed database is administered independently from all other databases.
- b. Nodes of the system cannot mirror the logical organization of companies or groups that need to maintain independence.
- c. Local administrators cannot control corresponding local data. Therefore, each database administrator's domain of responsibility is smaller and more manageable.
- d. Administrators cannot recover from isolated system failures independently from other nodes in the system.

✓ **Answer:** a. Each server participating in a distributed database is administered independently from all other databases.

14. A set of databases in a distributed system that can appear to users as a single data source is known as

- a. Distributed database
- b. Distributed processing
- c. Both A & B
- d. None of the above

✓ **Answer:** c. Both A & B

CH 1.2: DDBMS ARCHITECTURES

Contents

Reference Architecture	1
Client-Server Architecture	2
(i) Using DDBMS Primitives.....	2
(ii) Using Auxiliary Program and RPC (Remote Procedure Call)	3
Collaborating Server Architecture	4
Middleware Architecture	4
Peer-to-Peer (P2P) Architecture	5
Multi-DBMS/ Federated Database System Architecture.....	5
Transparency in Distributed Databases.....	7
Data Replication in Distributed Databases.....	10

Reference Architecture

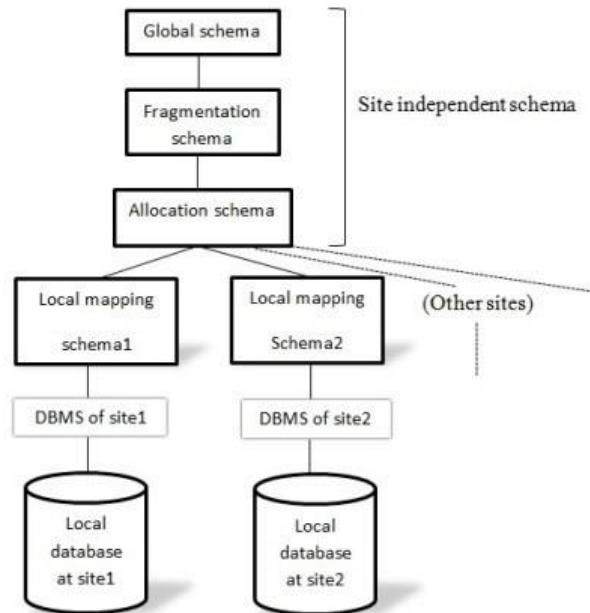
The reference architecture of a Distributed DBMS (DDBMS) defines the logical structure and functional components that enable distributed database management. It provides a standardized framework for designing and implementing DDBMSs. **Components of Reference Architecture:**

- **Global Schema (Conceptual Level):** Represents the overall logical view of the distributed database as a single entity other than showing the database is distributed at all.
Eg: Emp (EID, ENAME, DEPT)
- **Fragmentation Schema:** Defines how data in global schema is divided into fragments (horizontal, vertical, or mixed) and their placement in the distributed system.
Eg: Fragmenting the Employee global schema into:
 - **E1 based in Dept= 'Manager' Emp**
 - **E1 based in Dept= 'Sales Emp**
- **Allocation Schema:** Specifies where each fragment is stored (sites) across different nodes.
Eg: Allocation Schema: E1 at site 2,5 and E2 at site 1,3
- **Local Schemas (Internal Level):** Represents the physical storage and organization of data in each local database. Helps to identify the global relation schema for any local database relation schema.
- **External Schema (User Level):** Defines user views, specifying which part of the database a user or application can access.

Advantages:

- Provides a **structured approach** to designing DDBMS.
- Helps in **efficient query optimization** and transaction management.

- Supports **scalability** and **fault tolerance** by defining clear data distribution rules.



Client-Server Architecture

This architecture involves one or more **clients** that requests data and one or more **servers** that processes and responds to these requests. It is one of the most simple and common models used in DDBMS due to centralized server system. There are two variations:

(i) Using DDBMS Primitives

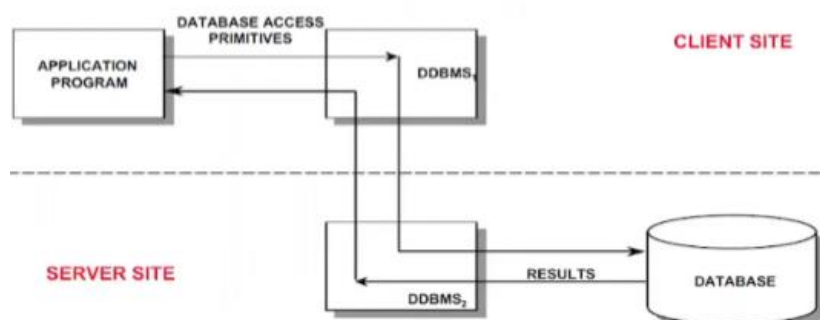
In this approach, clients interact directly with a Distributed DBMS that provides built-in support for distributed database operations. **How it works:**

- There are 2 sites: client site and server site
- The local server of DDBMS at client site routes the query to the server site.
- Clients send **structured queries** (SQL) to the DDBMS.
- The **DDBMS at server site handles query processing**, fragmentation, and data retrieval.
- It accesses the database, resolves the query and sends it back to the client site.

Features:

- Uses **standardized query languages (SQL)** with extensions for distribution.
- Query optimization and transaction management are **handled by the DDBMS itself**.
- High distribution transparency (because file names are global).
- Low efficiency since the answer travels 1 tuple at a time.

USING DDBMS PRIMITIVES



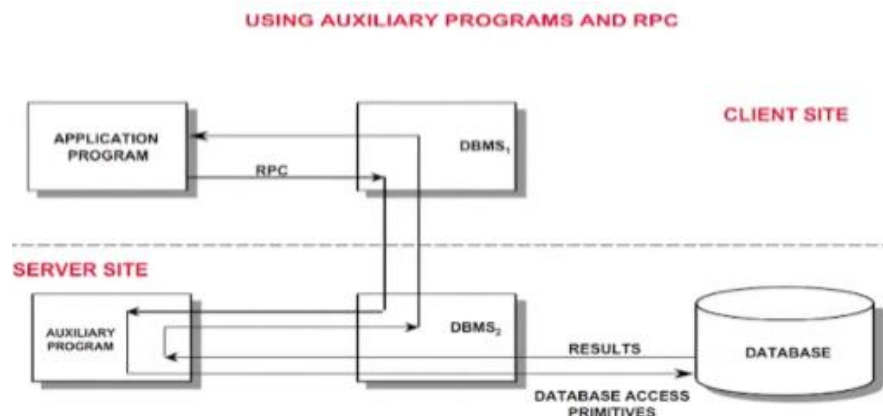
Advantages:

- Easier to manage as the DDBMS **automates** data distribution tasks.
- Provides **better performance** in distributed query execution.
- More **secure and reliable** since transaction control is centralized in the DDBMS.

(ii) Using Auxiliary Program and RPC (Remote Procedure Call)

This approach involves additional auxiliary programs to manage distributed queries and transactions instead of relying solely on the DDBMS. **How it works:**

- The application program at Clients site send requests to an **auxiliary program** that acts as an intermediary.
- The auxiliary program **interprets queries and manages communication** between clients and multiple servers.
- Remote Procedure Calls (RPC) enable **clients to execute procedures on remote servers** as if they were local.
- The auxiliary program assembles tuples into result sets improving transmission efficien



Advantages:

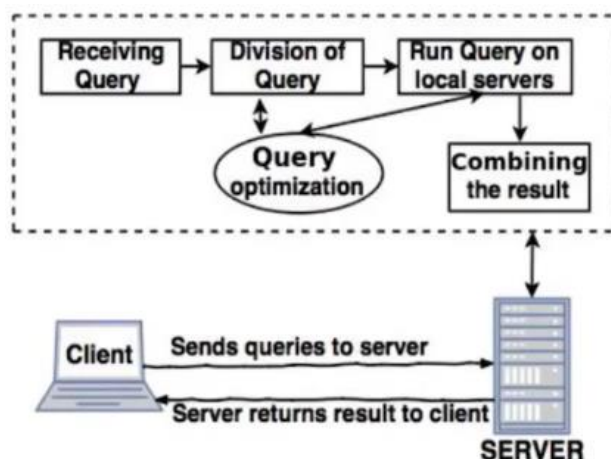
- Reduces the complexity of the DDBMS by delegating tasks to external programs.
- Provides **flexibility** as auxiliary programs can be customized for specific applications.
- RPC-based communication is **efficient** for distributed processing.

Feature	Reference Architecture	Client-Server (DDBMS Primitives)	Client-Server (Auxiliary Program & RPC)
Focus	Standardized structure for distributed DBMS	Direct interaction with DDBMS	Uses intermediary for distributed processing
Query Processing	Defined layers for query optimization	Handled by DDBMS	Managed by auxiliary program
Transaction Management	Defined structure for ACID properties	Centralized in DDBMS	Requires custom handling
Communication	Standardized architecture	Direct SQL queries	RPC-based calls
Flexibility	High (supports multiple models)	Medium (DDBMS-dependent)	Very High (customizable middleware)
Ease of Management	Complex but structured	Easier due to DDBMS automation	More effort needed for custom solutions

Collaborating Server Architecture

In this architecture, multiple database servers work together to process a single query or transaction efficiently. Instead of a single server managing everything, multiple servers **collaborate** to handle distributed data and transactions. **How it Works:**

- Each server has its own **local database** but can also access data from other servers.
- Upon receiving query server breaks the single query into multiple small queries and result is sent to the client.
- A **distributed query processor** manages queries across multiple servers.



Key Features:

- Supports **data distribution, consistency and concurrency across multiple sites**.
- Servers coordinate with each other **without a central controller**.
- Allows **high availability** and **fault tolerance**.

Advantages:

- **Scalability:** As more servers are added, performance improves.
- **Reliability:** If one server fails, others can still function.
- **Efficient Query Execution:** Queries can be executed in parallel across multiple servers.

Disadvantages:

- **Complex coordination** is required between servers.
- **Increased communication overhead** can impact performance.

Middleware Architecture

This architecture introduces an **intermediate layer (middleware)** between the client and the database servers. **How it Works:**

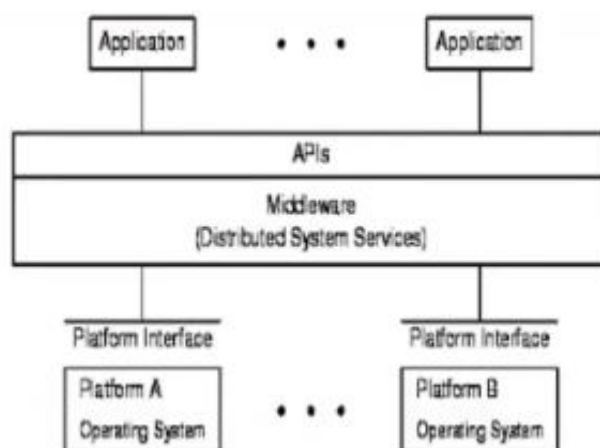
- Single query is executed in multiple servers.
- Clients send queries to the **middleware layer**.
- The middleware processes and optimizes the queries, then forwards them to the appropriate database servers.
- Middleware integrates data from multiple sources and presents a **unified view** to clients.

Key Features:

- Provides **location transparency**: Clients don't need to know where the data is stored.
- Supports **heterogeneous databases** (databases from different vendors).
- Offers **load balancing** and **query optimization**.

Advantages:

- **Improved interoperability:** Can integrate different types of databases.
- **Better performance:** Middleware optimizes query execution.
- **Security:** Middleware can enforce access controls and authentication.



Disadvantages:

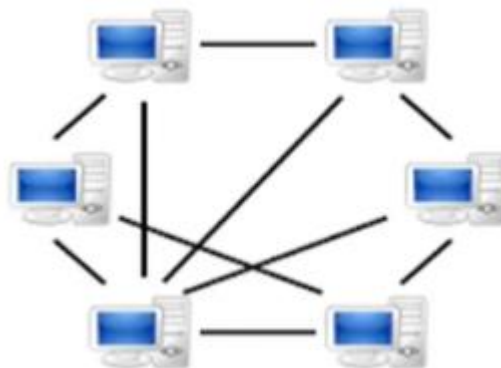
- **Additional processing delays** due to middleware overhead.
- **Complex implementation:** Needs careful configuration to ensure efficiency.

Peer-to-Peer (P2P) Architecture

In a **peer-to-peer** (P2P) architecture, all database nodes act as **equal peers**. There is no centralized control, and each node can function as both a client and a server.

How it Works:

- Each peer stores part of the database and can **process queries independently**.
- Peers communicate directly with each other to **share data and transaction information**.
- No **master-slave** relationship—each peer has equal responsibility.



Key Features:

- **Decentralized:** No central authority or control.
- **Self-organizing:** Peers can join or leave dynamically.
- **Replication and redundancy:** Data is often duplicated across peers to enhance availability.

Advantages:

- **Fault tolerance:** If one peer fails, others continue functioning.
- **Scalability:** New peers can be added easily.
- **Efficient resource utilization:** Workload is distributed across peers.

Disadvantages:

- **Data consistency issues:** Keeping data synchronized across peers is challenging.
- **Security concerns:** No central control makes access management difficult.
- **Higher communication overhead:** Each peer must communicate frequently with others.

Multi-DBMS/ Federated Database System Architecture

Multi-DBMS is designed to integrate multiple databases that may be **heterogeneous (from different vendors)** or **homogeneous (same vendor)**.

How it Works:

- A **federated database management system (FDBMS)** coordinates queries and transactions across multiple databases.
- Each database retains **local autonomy**—they continue operating independently.
- The system can be **loosely coupled (with minimal integration)** or **tightly coupled (with more central control over query execution)**.

The architecture has two major layers:

- A. Global (Multi-Database) Layer**
- B. Local (Individual DBMS) Layer**

1. Multi-Database View Level/Global External Schema.

- Represents multiple **user/application views** over multiple databases.
- Provides a **customized view** hiding system and distribution complexity.

2. Multi-Database Conceptual Level/Global Conceptual Schema (GCS).

- Describes the **logical structure** of the **entire federation**.
- Provides a **global schema** integrating the schemas of all local databases.
- Handles issues like **schema mapping, conflicts, and data integration**.
- Acts as a mediator between global views and local databases.

3. Multi-Database Internal Level/Global Internal Schema.

- Deals with the **physical representation** of the federated database.
- Manages the **distribution details, data fragmentation, replication, and communication** between databases.

4. Local Database View Level/ Local External Schema.

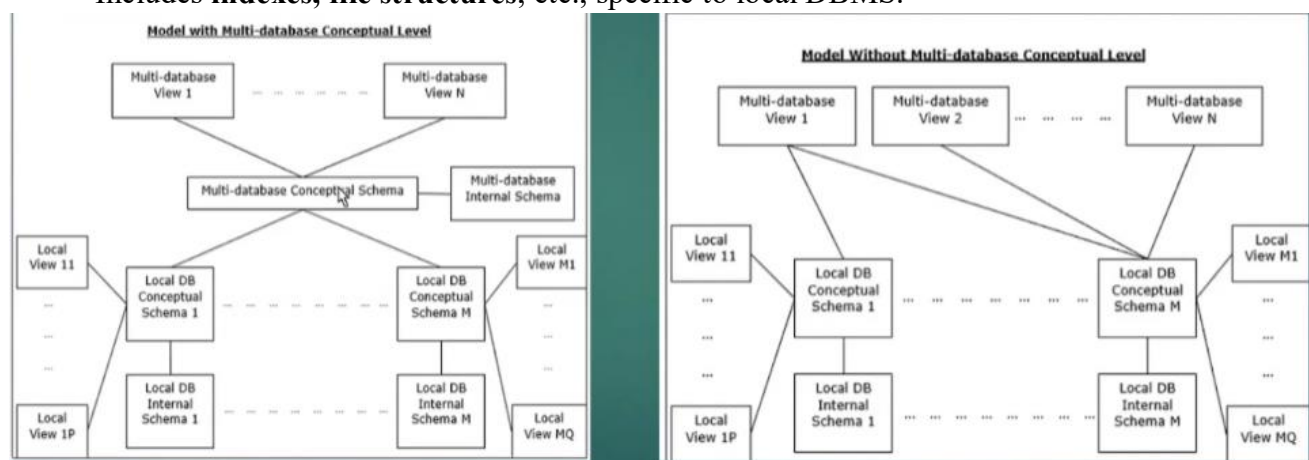
- Represents the **local user views** defined on each autonomous DBMS.
- Specific to individual databases — not aware of the global system.

5. Local Database Conceptual Level/Local Conceptual Schema.

- Defines the **logical structure** of data in each local database.
- Maintained by each local DBMS independently.

6. Local Database Internal Level/ Local Internal Schema.

- Describes **how data is stored** physically at each local site.
- Includes **indexes, file structures, etc.**, specific to local DBMS.



Advantages:

- **Flexibility:** Can integrate databases from different vendors.
- **No need for data migration:** Each database remains in its original format.
- **Better resource utilization:** Databases continue to function independently.

Disadvantages:

- **Query processing is complex:** Requires query transformation across different database types.
- **Performance overhead:** Due to data integration and coordination between databases.
- **Security challenges:** Managing access control across multiple databases is difficult.

Multi-database View Level - Depicts multiple user views comprising of subsets of the integrated distributed database.

Multi-database Conceptual Level - Depicts integrated multi-database that comprises of global logical multi-database structure definitions.

Multi-database Internal Level - Depicts the data distribution across different sites and multi-database to local data mapping.

Local database View Level - Depicts public view of local data.

Local database Conceptual Level - Depicts local data organization at each site.

Local database Internal Level - Depicts physical data organization at each site.

Architecture	Control Type	Scalability	Fault Tolerance	Heterogeneous DB Support	Complexity
Collaborating Server	Distributed	High	High	No	High
Middleware	Centralized Middleware	Medium	Medium	Yes	High
Peer-to-Peer (P2P)	Decentralized	Very High	Very High	No	Very High
Multi-DBMS	Semi-Decentralized	Medium	Medium	Yes	High

Transparency in Distributed Databases

- In distributed systems, the users are unaware of where the data are physically stored and how the data can be accessed at specific local site.
- User should be able to access the database exactly as if the system were local.
- Hiding all details from the users is called data transparency.
- Transparency allows a physically dispersed database to be managed as if it were a centralized database.
- Distribution transparency is the property of distributed databases by the virtue of which internal details of the distribution are hidden from the user.

Types of Transparency in DDBMS

1. **Location Transparency:** Users can access data without knowing where it is physically stored. The database system automatically determines the best site from which to retrieve data. **Example:** A user in Delhi queries for customer records, but the data is stored in Mumbai. The system fetches the data without requiring the user to specify the location.
2. **Naming Transparency:** Users can refer to data items using the same name system-wide, regardless of where the data is stored. **Example:** A table named `Customers` is accessed by the same name across all sites.
3. **Replication Transparency:** Users do not need to know whether multiple copies (replicas) of data exist. The DDBMS ensures that all replicas remain consistent and updated. **Example:** If

an inventory database is replicated at multiple warehouses, any update to stock levels at one warehouse is automatically reflected in all replicas.

4. **Fragmentation Transparency:** Users do not need to know if the database is divided into smaller parts (fragments) and stored at different locations. The system automatically reconstructs the data when needed. **Example:** A customer database may be fragmented based on region, with records for customers in North India stored in Delhi and those in South India stored in Bangalore, but users can access the full database without concern for fragmentation.
5. **Local Mapping Transparency:** Hides how data is stored at each local site; users work with a unified view. **Example:** A user accesses the Orders table without knowing its different storage formats at different sites.
6. **Transaction Transparency:** Ensures that a **distributed transaction** (a transaction involving multiple sites) maintains the **ACID** (Atomicity, Consistency, Isolation, Durability) properties. Users perceive transactions as if they are occurring in a single system. **Example:** A bank transfer between two accounts in different branches should appear as a single transaction even if it involves multiple databases.
7. **Performance Transparency:** Users receive optimized query responses regardless of the underlying distribution of data. The system ensures load balancing and efficient data retrieval. **Example:** A hotel booking system retrieves room availability quickly, even if data is stored across multiple cities.

Levels of Transparency: The 3 main levels of transparencies are:

- **Location Transparency**
- **Fragmentation Transparency**
- **Local Mapping Transparency**

Example

Consider an Emp table with the following schema: **Emp (ENAME, EDOB, EAddr, EDept, ESal)**

This employee data is horizontally fragmented based on location and distributed across three sites:

- Fragment E1: Employees in **Delhi**
- Fragment E2: Employees in **Mumbai**
- Fragment E3: Employees in **Kolkata**

- ⇒ Each fragment contains unique records, meaning no employee appears in more than one fragment.
- ⇒ Also, there is no replication — data is only stored at one site.
- ⇒ A user wants to list all employees whose date of birth is earlier than January 1, 1960.

Depending on the level of distribution transparency support we may examine three query cases

Case 1: the database supports fragmentation transparency

Case 2: the database supports location transparency

Case 3: the database supports local mapping transparency

Answer:

Case 1: Fragmentation Transparency

The user is unaware of the fragmentation. The system automatically accesses all fragments.

```
SELECT *  
FROM Emp  
WHERE EDOB < '1960-01-01';
```

✓ Only one query is needed. The DBMS handles fetching from all fragments (E1, E2, E3) automatically.

Case 2: Location Transparency

The user knows the table is distributed across sites but doesn't know the specific site locations.

```
SELECT * FROM E1  
WHERE EDOB < '1960-01-01'  
UNION  
SELECT * FROM E2  
WHERE EDOB < '1960-01-01'  
UNION  
SELECT * FROM E3  
WHERE EDOB < '1960-01-01';
```

✓ Here, the user queries all known locations, and the system handles the connection to each site.

Case 3: Local Mapping Transparency

The user must know exactly where the data is stored and how it's partitioned.

```
SELECT * FROM E1  
WHERE EDOB < '1960-01-01'  
UNION  
SELECT * FROM E2  
WHERE EDOB < '1960-01-01'  
UNION  
SELECT * FROM E3  
WHERE EDOB < '1960-01-01';
```

✓ Here, the user explicitly accesses all fragments (E1, E2, E3) with full awareness of the fragmentation and physical schema.

Data Transparency

Advantages:

- Users don't need to know data location
- Simplifies application development
- Supports data independence
- Offers a centralized view of data

Disadvantages:

- Complex system architecture
- Performance overhead
- High implementation cost

Data Replication

Advantages:

- Improves data availability
- Reduce network traffic
- Faster data access
- Enhances fault tolerance
- Enables load distribution

Disadvantages:

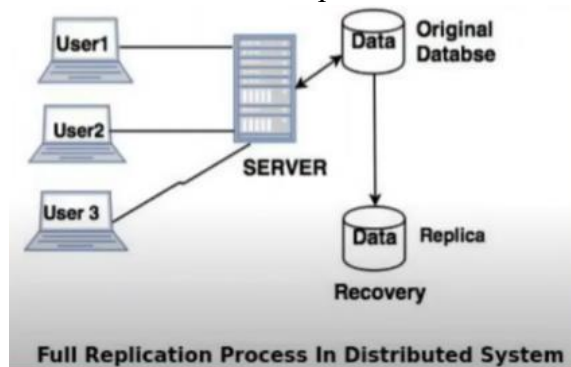
- Complex update management
- More storage needed
- Risk of data inconsistency
- Higher maintenance requirements

Data Replication in Distributed Databases

Data Replication is the process of storing multiple copies of the same data at different locations to improve reliability, availability, and performance. It helps ensure **fault tolerance**, meaning that if one site fails, another site can serve the same data. **Types of Data Replication**

1. **Full Replication:** Every site contains a complete copy of the entire database. Provides high availability and fault tolerance but requires significant storage and synchronization efforts.

Example: A global banking system may fully replicate customer data across all branches so transactions can proceed even if one branch server fails.



Full Replication

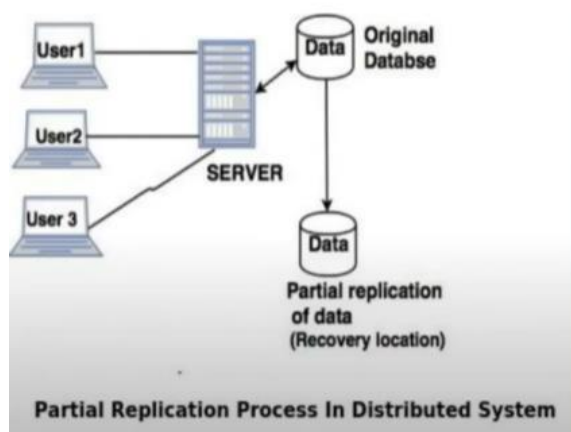
Advantages of full replication

- High availability of data, as database is available to almost every location.
- Faster execution of queries.

Disadvantages of full replication

- Concurrency control is difficult to achieve in full replication.
- Update operation is slower.

2. **Partial Replication:** Only selected parts of the database are replicated at different sites. Reduces storage overhead and improves efficiency while maintaining redundancy. **Example:** An e-commerce platform may store customer order history at a central location but replicate only current orders to regional warehouses.



Partial Replication

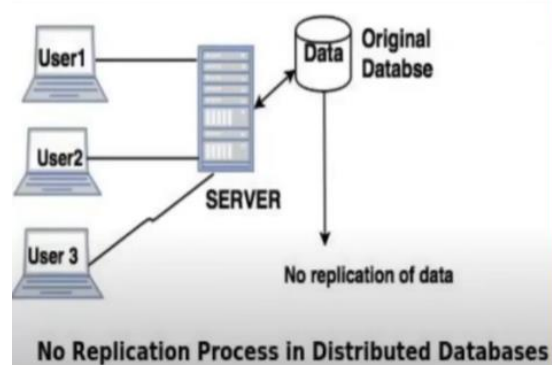
Advantages of Partial replication

- To provide a consistent copy of data across all the database nodes.
- To increase the availability of data.
- The reliability of data is increased through data replication.
- Data Replication supports multiple users and gives high performance

Disadvantages of Partial replication

- More storage space is needed as storing the replicas of same data at different sites consumes more space.
- Data Replication becomes expensive when the replicas at all different sites need to be updated.
- Maintaining Data consistency at all different sites involves complex measures.

3. **No Replication** Data is not duplicated, and each piece of data exists at only one site. Reduces storage requirements but increases the risk of unavailability if a site goes down. **Example:** A university database storing student records in a single centralized location instead of replicating across different campuses.



No Replication

Advantages of no replication

- Concurrency can be minimized.
- Easy recovery of data.

Disadvantages of no replication

- Poor availability of data.
- Slows down the query execution process, as multiple clients are accessing the same server.

CH 2.1: DISTRIBUTED DATABASE DESIGN

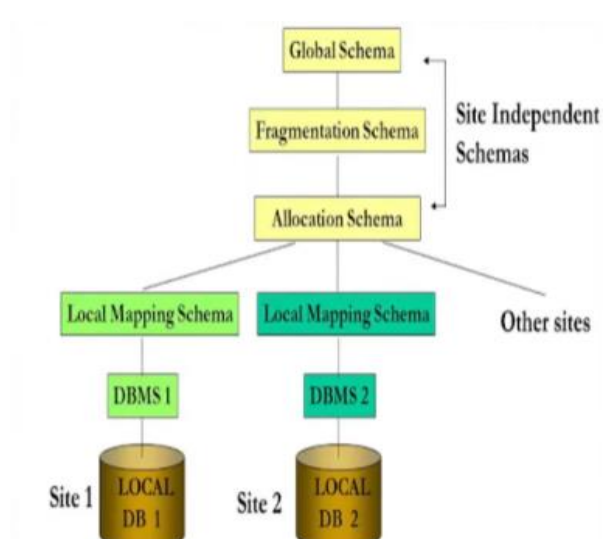
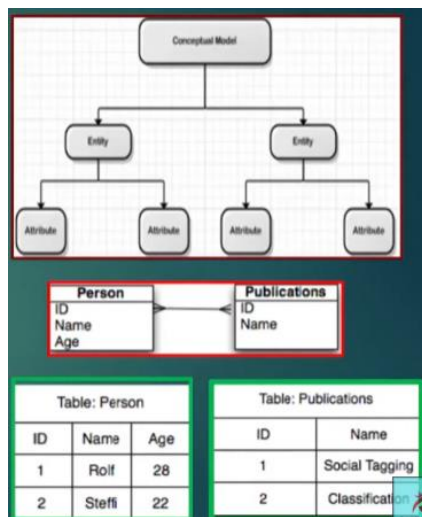
Contents

Contents	1
Alternative Design Approaches	1
Loosely and Tightly Coupled System	2
Fragmentation.....	3
Correctness Rules in Fragmentation	3
Horizontal Fragmentation:	4
Types of Horizontal Fragmentation:	4
1. Primary Fragmentation.....	4
2. Derived Fragmentation.....	4
Min term predicates	5
Vertical Fragmentation.....	6
Mixed (Hybrid) Fragmentation	7
Data Allocation.....	9

Alternative Design Approaches

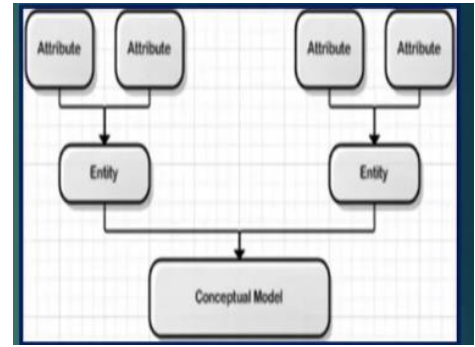
1. Top-Down Approach

- The design starts from a **global schema**.
- First, the **entire system requirements** are analyzed.
- A **global conceptual schema** is designed as if the system is centralized.
- Then, it is **partitioned** and **allocated** to different sites.
- Suitable when the organization has a clear view of global data needs.
- Ensures **data integration and consistency** from the beginning.
- **Example:** Design a company-wide employee database first, then divide it among regional offices.



2. Bottom-Up Approach

- Begins with **existing local databases** at various sites.
- Each site's **schemas are analyzed** and documented.
- Then, these local schemas are **integrated** into a **global schema**.
- Suitable when the system has **autonomous databases** already in use.
- More complex due to **schema mismatches and conflicts**.
- **Example:** Combine separate HR, finance, and sales databases from different branches into one global view.



Loosely and Tightly Coupled System

Loosely Coupled and Tightly coupled System

Distributed Systems
 " A Distributed System is a collection of independent computers that appears to its users as a single coherent system "

12

Tightly Coupled Systems

- Systems with a single system wide memory
- Parallel Processing System , SMMP (shared memory multiprocessor systems)

Loosely Coupled System

- Distributed Memory Systems (DMS)
- Communication via Message Passing

1. Tightly coupled is a concept of system design and computing where every hardware & software components that are linked together in such manner that each component is dependent upon each other.

2. Key point to be noticed is it shares the memory across the CPU's. Communication will be established using shared memory.

1. Every module is somewhat independent of each other

2. In a loosely coupled system, hardware and software may interact but they are not dependent on each other.

3. Loosely Coupled Systems are actually Distributed Systems. Because System which does not share memory with other systems and have their own clock (indicates how fast the CPU can run). It is a Distributed system

[SUBSCRIBE](#)

Feature	Tightly Coupled System	Loosely Coupled System
Definition	Systems with shared memory and close integration	Systems with distributed memory and components
Components	Closely connected	Independent and autonomous
Communication	High-speed via shared memory	Message passing over network
Performance	Faster	Slower compared to tightly coupled
Scalability	Less scalable	Highly scalable
Fault Tolerance	Low – failure affects entire system	High – failure isolated
Example	Multi-core processors	Distributed systems (e.g., cloud)
Maintenance	Harder to modify	Easier to upgrade or replace parts

Fragmentation

- Fragmentation is the process of **dividing a database into smaller pieces** (fragments) that can be **stored at different sites** in a distributed database system.
- Each fragment is stored at one or more sites and is **logically related to the original database**.
- Fragmentation should be carried out in such a way that **reconstruction** of original database from the fragments is possible.
- Fragmentation helps in improving **efficiency, parallelism, and locality of data access**.

Correctness Rules in Fragmentation

When fragmenting a relation in distributed databases, certain **correctness rules** must be satisfied to ensure:

- We should not lose data because of fragmentation
- We should not get redundant data because of fragmentation.

To ensure these properties we need to verify if fragmentation is correctly performed or not. Due to this we have the following correctness rules:

1. Completeness

- Every data item that is in the original relation must appear in at least one fragment.
- No tuple from the original relation should be missing after fragmentation.

$$R_1, R_2, \dots, R_n \in R$$

- **Example:** If the original table EMP has 100 tuples, then the total of all tuples across the fragments must also be 100.

2. Reconstruction (Recomposability)

- It must be possible to reconstruct the original relation by applying a relational operation (usually **UNION** for horizontal fragmentation and **JOIN** for vertical fragmentation) on the fragments.
- The system must be able to retrieve the original relation without any loss or duplication of data.
 - **Horizontal Fragmentation:** $R = R_1 \cup R_2 \cup \dots \cup R_n$
 - **Vertical Fragmentation:** $R = \pi_{a1}(R_1) \bowtie \pi_{a2}(R_2) \bowtie \dots \bowtie \pi_{an}(R_n)$

3. Disjointness (for horizontal fragmentation)

- The data in each fragment must be **mutually exclusive** — no data item should appear in more than one fragment.
- This ensures that each tuple belongs to exactly one fragment, preventing data duplication.
- If a table R is partitioned into fragments $R_1, R_2, \dots, R_n$ then disjointness insists the following:

$$R_1 \cap R_2 \cap \dots \cap R_n = \emptyset$$

- **Example:** If EMP1 contains employees from Delhi, then EMP2 (e.g., Mumbai) should not include any Delhi employee.

Degree of Fragmentation: The degree of fragmentation refers to the extent to which a database relation (table) is divided into fragments. It reflects how many fragments a relation has after applying a fragmentation strategy.

Horizontal Fragmentation:

- Divides a relation (table) **row-wise** based on certain **conditions or predicates**.
- Each fragment contains a subset of **tuples (rows)**.
- Every fragment has all the columns and a subset of origin relation.
- All fragments are stored in different sites and each fragment are distinct tuples.
- We can distribute the pieces to the sites of distributed database however we like and reassemble them.

Consider a table:

```
EMPLOYEE(EID, Name, City, Salary)
```

- Fragment 1: All employees in Delhi

```
EMP_DELHI = SELECT * FROM EMPLOYEE WHERE City = 'Delhi';
```

- Fragment 2: All employees in Mumbai

```
EMP_MUMBAI = SELECT * FROM EMPLOYEE WHERE City = 'Mumbai';
```

Types of Horizontal Fragmentation:

1. **Primary Fragmentation:** Primary fragmentation is the initial and independent division of a global relation (table) based on either horizontal, vertical, or mixed fragmentation.
 - Applied directly on the original global table.
 - Each primary fragment is complete and disjoint.
 - Used to distribute data based on access patterns or location-specific requirements.

Example:

```
CUSTOMER(CustID, Name, City)
```

Horizontal primary fragmentation based on `City` directly derived from the global `CUSTOMER` table:

- `CUST_Mumbai = SELECT * FROM CUSTOMER WHERE City = 'Mumbai'`
- `CUST_Delhi = SELECT * FROM CUSTOMER WHERE City = 'Delhi'`

2. **Derived Fragmentation:** Derived fragmentation occurs when a relation is fragmented based on the fragmentation of another related relation, typically through foreign key relationships.
 - Used when two tables are logically related.
 - Maintains referential integrity.
 - Ensures co-location of related data to reduce join costs.

Suppose we have two relations:

1. `CUSTOMER(CustID, Name, City)`
2. `ORDERS(OrderID, CustID, Product, Amount)`

If `CUSTOMER` is horizontally fragmented by city:

- `CUST_Mumbai`
- `CUST_Delhi`

Then `ORDERS` can be **derived fragmented** using `CustID` from the already fragmented `CUSTOMER` table:

- `ORDERS_Mumbai = SELECT * FROM ORDERS WHERE CustID IN (SELECT CustID FROM CUST_Mumbai)`
- `ORDERS_Delhi = SELECT * FROM ORDERS WHERE CustID IN (SELECT CustID FROM CUST_Delhi)`

This way, each `ORDERS` fragment corresponds to the respective `CUSTOMER` fragment.

Min term predicates

Primary Horizontal Fragmentation (PHF)

Primary Horizontal Fragmentation is about fragmenting a single table horizontally (row wise) using a set of simple predicates (conditions).

25

What is simple predicate?

Given a table R with set of attributes $[A_1, A_2, \dots, A_n]$, a simple predicate P_i can be expressed as follows:

$P_i : A_i \theta \text{ Value}$

Where θ can be any of the symbols in the set $\{=, <, >, \leq, \geq, \neq\}$, value can be any value stored in the table for the attributed A_i .

Acno	Balance	Branch_Name
A101	5000	Mumbai
A103	10000	New Delhi
A104	2000	Chennai
A102	12000	Chennai
A110	6000	Mumbai
A115	6000	Mumbai
A120	2500	New Delhi

Figure 1: Account table

Example 1: for the table Account, if simple conditions are, $\text{Balance} < 10000$, $\text{Balance} \geq 10000$, then, Set of simple predicates $P_1 = \{\text{Balance} < 10000, \text{Balance} \geq 10000\}$

Example 2: if simple conditions are, $\text{Branch_name} = \text{'Chennai'}$, $\text{Branch_name} = \text{'Mumbai'}$, $\text{Balance} < 10000$, $\text{Balance} \geq 10000$, then, Set of simple predicates $P_2 = \{\text{Branch_name} = \text{'Chennai'}$, $\text{Branch_name} = \text{'Mumbai'}$, $\text{Balance} < 10000$, $\text{Balance} \geq 10000\}$

SUBSCRIBE

Min-term Predicate

Min-term predicate which can be written as follows;

Min-term predicate, $M_i = P_1 \wedge P_2 \wedge P_3 \wedge \dots \wedge P_n$

- Here, P_1 means both P_1 or $\neg(P_1)$, P_2 means both P_2 or $\neg(P_2)$, and so on. Using the conjunctive form of various simple predicates in different combination, we can derive many such min-term predicates.
- For **Example 1:** for the table Account, if simple conditions are, $\text{Balance} < 10000$, $\text{Balance} \geq 10000$, then,
 - We can derive set of min-term predicates using the rules stated above as follows;
 - We will get 2^n min-term predicates, where n is the number of simple predicates in the given predicate set. For P_1 , we have 2 simple predicates. Hence, we will get 4 (2^2) possible combinations of min-term predicates as follows;
 - $m_1 = \{\text{Balance} < 10000 \wedge \text{Balance} \geq 10000\}$
 - $m_2 = \{\text{Balance} < 10000 \wedge \neg(\text{Balance} \geq 10000)\}$
 - $m_3 = \{\neg(\text{Balance} < 10000) \wedge \text{Balance} \geq 10000\}$
 - $m_4 = \{\neg(\text{Balance} < 10000) \wedge \neg(\text{Balance} \geq 10000)\}$

Our next step is to choose the min-term predicates which can satisfy certain conditions to fragment a table, and eliminate the others which are not useful.

- $\text{Account1} = \sigma_{\text{Balance} < 10000 \wedge \text{Balance} \geq 10000}(\text{Account})$
 - which can be written in equivalent SQL query as,
 $\text{Account1} \leftarrow \text{SELECT } * \text{ FROM account WHERE balance} < 10000 \text{ AND balance} \geq 10000;$
- $\text{Account2} = \sigma_{\text{Balance} < 10000 \wedge \neg(\text{Balance} \geq 10000)}(\text{Account})$
 - which can be written in equivalent SQL query as,
 $\text{Account2} \leftarrow \text{SELECT } * \text{ FROM account WHERE balance} < 10000 \text{ AND NOT balance} \geq 10000;$
where NOT balance ≥ 10000 is equivalent to balance < 10000 .
- $\text{Account3} = \sigma_{\neg(\text{Balance} < 10000) \wedge \text{Balance} \geq 10000}(\text{Account})$
 - which can be written in equivalent SQL query as,
 $\text{Account3} \leftarrow \text{SELECT } * \text{ FROM account WHERE NOT balance} < 10000 \text{ AND balance} \geq 10000;$
where NOT balance < 10000 is equivalent to balance ≥ 10000 .
- $\text{Account4} = \sigma_{\neg(\text{Balance} < 10000) \wedge \neg(\text{Balance} \geq 10000)}(\text{Account})$
 - which can be written in equivalent SQL query as,
 $\text{Account4} \leftarrow \text{SELECT } * \text{ FROM account WHERE NOT balance} < 10000 \text{ AND NOT balance} \geq 10000;$

27

■ The first query for fragment Account1 (min-term predicate m1) is invalid as any record in a table cannot have two values for any attribute in one record. That is, the condition $(Balance < 10000 \wedge Balance \geq 10000)$ requires that the value for balance must both be less than 10000 and greater and equal to 10000, which is not possible. Hence the condition violates and can be eliminated.

■ For fragment Account2 (min-term predicate m2), the condition is $(balance < 10000 \text{ and } balance < 10000)$ which ultimately means $balance < 10000$ which is correct.

■ Likewise, fragment Account3 is valid and Account4 must be eliminated.

Account1 = $\sigma_{Balance < 10000 \wedge Balance \geq 10000}(Account)$
 which can be written in equivalent SQL query as,
 Account1 $\leftarrow$ SELECT * FROM account WHERE balance < 10000 AND balance $\geq$ 10000;

Account2 = $\sigma_{Balance < 10000 \wedge \neg(Balance \geq 10000)}(Account)$
 which can be written in equivalent SQL query as,
 Account2 $\leftarrow$ SELECT * FROM account WHERE balance < 10000 AND NOT balance $\geq$ 10000;
 where NOT balance $\geq$ 10000 is equivalent to balance < 10000.

Account3 = $\sigma_{\neg(Balance < 10000) \wedge Balance \geq 10000}(Account)$
 which can be written in equivalent SQL query as,
 Account3 $\leftarrow$ SELECT * FROM account WHERE NOT balance < 10000 AND balance $\geq$ 10000;
 where NOT balance < 10000 is equivalent to balance $\geq$ 10000.

Account4 = $\sigma_{\neg(Balance < 10000) \wedge \neg(Balance \geq 10000)}(Account)$
 which can be written in equivalent SQL query as,
 Account4 $\leftarrow$ SELECT * FROM account WHERE NOT balance < 10000 AND NOT balance $\geq$ 10000;

SELECT * FROM account WHERE balance < 10000;

Account ₂		
Acno	Balance	Branch_Name
A101	5000	Mumbai
A104	2000	Chennai
A120	2500	New Delhi
A110	6000	Mumbai
A115	6000	Mumbai

Account ₃		
Acno	Balance	Branch_Name
A103	10000	New Delhi
A102	12000	Chennai

Vertical Fragmentation

Divides a relation **column-wise**, keeping the **primary key** in all fragments to allow reconstruction via join.

- Fragment 1: EMP_NAMES = PROJECT EID, Name FROM EMPLOYEE;
- Fragment 2: EMP_DETAILS = PROJECT EID, City, Salary FROM EMPLOYEE;

id	Name	Sex	Age
1	Sebastian	M	3
2	John	M	5
3	Garry	M	8
4	Louis	F	7

Original Table

Vertical Fragmentation

id
1
2
3
4

Name
Sebastian
John
Garry
Louis

Sex
M
M
M
F

Age
3
5
8
7

Two approaches in vertical fragmentation:

1. Grouping (Bottom up approach)

- Grouping refers to **combining related attributes** together into fragments.
- To optimize query performance by keeping frequently accessed attributes together.
- Example:**

Consider relation EMP(EID, EName, Dept, Salary, Phone) We can group:

- Fragment 1: EMP1(EID, EName, Dept) — for HR-related queries
- Fragment 2: EMP2(EID, Salary, Phone) — for payroll-related queries

Each fragment must include the primary key (EID) to enable reconstruction via **natural join**.

2. Splitting (Top down approach)

- Splitting refers to **dividing a relation's attributes into smaller sets** across different fragments.
- To distribute attributes that are used independently or to reduce data transfer.
- Example:**

From the above relation, you may split into:

- Fragment 1: EMP1(EID, EName)
- Fragment 2: EMP2(EID, Dept)
- Fragment 3: EMP3(EID, Salary)
- Fragment 4: EMP4(EID, Phone)

This is a **fine-grained** approach, useful when queries typically access only a subset of attributes.

Example of Vertical Fragmentation:

Eid	Ename	Salary	Bonus	Project	Experience
1	ABC	50,000	5000	P ₁	5
2	DEF	40,000	7000	P ₂	3
3	GHI	45,000	8000	P ₃	2

We will make fragments based on relevant information

- Primary key is included in every fragment.
- **Fragment1** - Basic Employee Details(Eid,Ename,Salary)
- **Fragment2** - Bonus on Projects(Eid,Project,Bonus)

Eid	Ename	Salary
1	ABC	50,000
2	DEF	40,000
3	GHI	45,000

Fragment1

Eid	Bonus	Project
1	5000	P ₁
2	7000	P ₂
3	8000	P ₃

Fragment2

Reconstruction

Reconstruction of vertical fragmentation is performed by using **Full Outer Join** operation on fragments.

- select * from
fragment 1
NATURAL JOIN
fragment 2

Eid	Ename	Salary
1	ABC	50,000
2	DEF	40,000
3	GHI	45,000

Fragment1

Eid	Bonus	Project
1	5000	P ₁
2	7000	P ₂
3	8000	P ₃

Fragment2

Mixed (Hybrid) Fragmentation

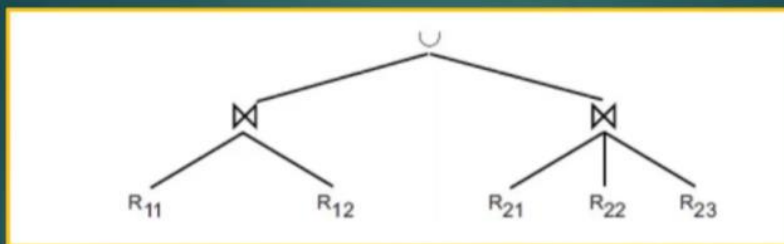
A combination of **horizontal and vertical fragmentation**.

- At first, generate a set of horizontal fragments; then generate vertical fragments from one or more of the horizontal fragments.
- At first, generate a set of vertical fragments; then generate horizontal fragments from one or more of the vertical fragments.

- Consider a relation: `Employee(EID, EName, EDept, ESalary, ELocation)`
- Step 1 – Horizontal Fragmentation:**
Fragment based on `ELocation`
 - `Emp_Delhi` : employees from Delhi
 - `Emp_Mumbai` : employees from Mumbai
- Step 2 – Vertical Fragmentation** (applied on `Emp_Delhi`):
 - `Emp_Delhi_1(EID, EName)`
 - `Emp_Delhi_2(EID, EDept, ESalary)`
- Final Result: Combination of horizontal and vertical fragments — a **hybrid** fragmentation strategy.

Reconstruction of Hybrid Fragmentation

39



The original relation in hybrid fragmentation is reconstructed by performing UNION and FULL OUTER JOIN.

1. Horizontal Fragmentation

Advantages:

- Queries accessing specific rows are faster.
- Easier to distribute data by location.

Disadvantages:

- Joins across fragments are expensive.
- May lead to data duplication if not disjoint.

2. Vertical Fragmentation

Advantages:

- Improves performance for queries accessing few columns.
- Reduces the size of tuples.

Disadvantages:

- Requires joins to access complete tuples.
- Increases complexity of query processing.

3. Hybrid Fragmentation

Advantages:

- Combines benefits of both horizontal and vertical fragmentation.
- Greater flexibility and optimization for complex access patterns.

Disadvantages:

- Most complex to implement and manage.
- Higher overhead for reconstruction and maintenance.

Data Allocation

- A database that consists of two or more data files allocated at different sites on a computer network is known as data location.
- The main objective of data allocation in distributed database is to be placed the data segment at different sites in such a way so that the total data transfer cost can be minimum while executing a set of queries.

Data Allocation Strategies

1. Centralized Allocation

- All data stored at a single site.
- Simple to manage but not fault-tolerant.
- Low data redundancy.

2. Partitioned Allocation

- Data divided into fragments and stored at different sites.
- Can be horizontal, vertical, or hybrid fragmentation.
- Reduces data access time for local users.

3. Replicated Allocation

- Copies of data (or fragments) are stored at multiple sites.
- Improves availability and fault tolerance.
- Increases update overhead and consistency complexity.

Factors Affecting Data Allocation

- Frequency of data access by different sites.
- Communication cost between sites.
- Query and transaction patterns.
- Site reliability and availability.
- Data security and privacy concerns.
- Network latency and bandwidth.

Information Requirements for Data Allocation

a. Database Information:

- Selectivity of fragments
- Size of a fragment

b. Application Information:

- Number of read accesses of a query to a fragment
- Number of update accesses of a query to a fragment
- A matrix indicating which queries update which fragments
- A similar matrix for retrievals
- Originating site of each query

c. Site Information:

- Unit cost of storing data at a site
- Unit cost of processing at a site

d. Network Information:

- Communication cost/frame between two sites
- Frame size (Frames are units of data in the Link Layer, e.g., WiFi, Bluetooth, Ethernet, etc.)

Allocation Model

- It has two components: A) Processing Component
B) Transmission Component

A) Processing Component:

Three factors :

1. Access cost(AC)
2. Integrity Enforcement(IE)
3. Concurrency Control cost(CC)

1. Access cost(AC)

$$AC_i = \sum_{\text{all sites}} \sum_{\text{all fragments}} (U_{ij} * UR_{ij} + r_{ij} * RR_{ij}) * LPc_k * X_{jk}$$

U_{ij} = equals 1 when update else 0 on f_j

UR_{ij} = no of update acesses

r_{ij} = equals 1 when read else 0 on f_j

RR_{ij} = no of read access

LPc_k = cost of access at a site K

X_{jk} = This ensures that f_j is stored at site K

The decision variable is x_{jk} , which is defined as $x_{jk} = \begin{cases} 1 & \text{if the fragment } f_j \text{ is stored at site } k \\ 0 & \text{otherwise} \end{cases}$

Integrity enforcement and concurrency control algorithm execution time costs are IE & CC respectively

B) Transmission Component

- It has two components-

1. Processing update cost
2. Processing retrieval cost

- In update queries it is necessary to inform all the sites where replicas exist,
- In addition, at the end of an update request, there is no data transmission back to the originating site other than a confirmation message,

1. Processing update cost

$$TCU_i = \sum_{\text{All sites}} \sum_{\text{All Fragments}} \text{update message cost} + \sum_{\text{All sites}} \sum_{\text{All Fragments}} \text{Acknowledge cost}$$

2. Processing Retrieval cost

- ✓ In retrieval queries, it is sufficient to access only one of the copies.
- ✓ The equation states that among all the sites with copies of the same fragment, only the site that yields the minimum total transmission cost should be selected for the execution of the operation.

$$TCR_i = \sum_{\text{All sites}} \min_{\text{all sites}} (\text{cost of retrieval command} + \text{cost of sending back the result})$$

Now the transmission cost function for query q_i can be specified as

Transmission cost function

$$TC_i = TCU_i + TCR_i$$

CH 2.2: SEMANTICS DATA CONTROL

Contents

Database Administrator (DBA)	1
Authentication	2
Access Rights (Authorization)	2
Semantic Integrity Constraints	3
View Management	4

Database Administrator (DBA)

DBA is a specialized IT professional responsible for **managing, maintaining, securing, and optimizing** databases in an organization. The DBA ensures that databases are **available, consistent, perform well**, and are **secure** from unauthorized access.

Key Roles and Responsibilities:

1. **Database Design:** Define schema structure, data types, relationships, and constraints.
2. **Installation & Configuration:** Install and set up DBMS software with proper connectivity.
3. **User Management:** Manage user roles and assign database access privileges.
4. **Backup and Recovery:** Schedule backups and plan for disaster recovery.
5. **Performance Tuning:** Optimize queries and indexes to boost performance.
6. **Security Management:** Protect data through access controls and encryption.
7. **Monitoring and Maintenance:** Track database health and apply regular updates.
8. **Data Integrity and Consistency:** Ensure data remains valid with rules and constraints.
9. **Capacity Planning:** Predict and plan for growing storage needs.
10. **Support and Troubleshooting:** Assist users and fix database-related issues.

Skills Required:

1. Proficiency in **SQL** and **DBMS tools**
2. Knowledge of **operating systems** and **networking**
3. Understanding of **data modeling** and **design principles**
4. Skills in **performance tuning, backup and recovery tools**
5. Awareness of **cybersecurity** principles

Common DBMS Tools a DBA Works With:

1. Oracle
2. Microsoft SQL Server
3. MySQL/MariaDB
4. PostgreSQL
5. MongoDB
6. IBM DB2

Authentication

Authentication is the process of verifying the identity of users who are trying to access the database.

Example: A student portal requires login credentials before allowing access to academic records.

- Ensures that only **authorized users** can log in.
- Common methods include **usernames & passwords**, **biometric scans**, or **two-factor authentication**.
- Helps prevent **unauthorized access** and protects sensitive data.

Authentication can be enforced in two levels -

Controlling Access to Client Computer - At this level, user access is restricted while login to the client computer that provides **user-interface** to the database server. The most common method is a **username/password** combination.

Controlling Access to the Database Software - At this level, the database administrator assigns some **credentials**(testimonials or certified documents showing that a person is entitled to credit)to the user. The user gains access to the database using these credentials. One of the methods is to create a **login account** within the database server.

Access Rights (Authorization)

Access rights define **what operations** a user is allowed to perform once authenticated. **Example:** In a hospital database, a doctor can view and update patient records, while a receptionist can only view them.

- Specifies **read, write, update, delete**, or **admin** rights.
- Can be assigned to **individual users or user groups**.
- Implemented through **roles, privileges, and permissions**.

Access Rights

✓ A user's access rights **refers to the privileges** that the user is given regarding DBMS operations such as the rights to create a table, drop a table, add/delete/update tuples in a table or query upon the table.

✓ In distributed environments, since there are **large number of tables and yet larger number of users**, it is not feasible to assign individual access rights to users. So, DDBMS defines **certain roles**. A role is a construct with certain privileges within a database system.

For example, the following SQL statements create a role "Accountant" and then assigns this role to user "ABC".

```
CREATE ROLE ACCOUNTANT;  
GRANT SELECT, INSERT, UPDATE ON EMP_SAL TO ACCOUNTANT;  
GRANT INSERT, UPDATE, DELETE ON TENDER TO ACCOUNTANT;  
GRANT INSERT, SELECT ON EXPENSE TO ACCOUNTANT;  
COMMIT;  
GRANT ACCOUNTANT TO ABC;  
COMMIT;
```

Semantic Integrity Constraints

Semantic Integrity Control ensures that the data stored in a database is **logically correct** and meaningful according to the rules and constraints of the application domain. It goes beyond simple structural correctness and focuses on the **validity of data values and relationships** based on business logic.

Types of Semantic Integrity Constraints

1. Data Type Integrity Constraint

- Ensures that each column in a table contains data of the correct **predefined type** (e.g., integer, varchar, date).
- Prevents insertion of invalid data types into columns.
- Example: A column defined to store age must only accept integers.

```
CREATE TABLE Employees (  
    emp_id INT,  
    name VARCHAR(100),  
    salary DECIMAL(10, 2) -- Ensures only numeric values with 2 decimal places  
);
```

This ensures `emp_id` is always an integer, `name` is a string, and `salary` is a decimal.

2. Entity Integrity Constraint

- Ensures that every table has a **primary key** and that this key is **unique and not null**.
- Maintains the **uniqueness** of each record in a table.
- Example: A student ID in a student table cannot be null or duplicated.

```
CREATE TABLE Students (  
    student_id INT PRIMARY KEY,  
    name VARCHAR(50)  
);
```

Here, `student_id` is a **primary key** and must be **unique and not NULL**.

3. Referential Integrity Constraint

- Ensures that **foreign keys** in a table correctly reference **primary keys** in another table.
- Maintains consistency between related tables.
- Example: A foreign key in the 'enrollments' table referencing 'student_id' in the 'students' table must match a valid student ID.

```
CREATE TABLE Courses (
    course_id INT PRIMARY KEY,
    course_name VARCHAR(100)
);

CREATE TABLE Enrollments (
    student_id INT,
    course_id INT,
    FOREIGN KEY (course_id) REFERENCES Courses(course_id)
);
```

This ensures that every `course_id` in `Enrollments` must exist in the `Courses` table.

View Management

View management is responsible for creating, storing, and maintaining **views** — virtual tables based on SQL queries. Views provide users with a **customized, simplified, or restricted** perspective of the database, enhancing **security, access control, and data abstraction**.

1. Views in Centralized DBMS

- **Definition:** A view is a virtual table generated by a query on base tables in the centralized database.
- **Purpose:** Helps restrict access to sensitive data by showing only selected columns/rows.
- **Example:**

```
CREATE VIEW EmployeeView AS
SELECT name, department FROM Employees WHERE role = 'Staff';
```

2. Updates Through Views

- **Update Possibility:** Simple views (based on a single table, without GROUP BY, JOIN, or aggregation) can usually be updated.
- **Restrictions:** Views involving multiple tables, joins, group functions, or DISTINCT are **not** usually updatable.
- **INSTEAD OF Triggers** (in some DBMSs): Used to handle updates on non-updatable views.
- **Example of updatable view:**

```
CREATE VIEW ActiveUsers AS
SELECT id, name FROM Users WHERE status = 'active';
```

3. Views in Distributed DBMS

- **Definition:** Views in distributed DBMSs may reference data from **multiple remote sites**.
- **Use:** Enable **location transparency** and **customized data access**.
- **Challenges:**
 - Update propagation across sites is complex.
 - View materialization and consistency become critical in a distributed setting.
- **Solution:** Use **global views** (defined over local schemas) and ensure **semantic correctness** through metadata synchronization and control mechanisms.

CH 2.3: QUERY PROCESSING

Contents

Query Processing	1
Characteristics of Query Processors	1
Layers of Query Processing	2
Challenges in Distributed Environments.....	3
1. Query decomposition	3
A. Normalization	4
B. Analysis	5
C. Elimination of Redundancy	7
D. Rewriting	7
2. Data Localization	8
Translating a Global Query	8
3. Global Query Optimization	10
4. Local Optimization.....	10
Query Processing vs Query Optimization.....	11

Query Processing

Query Processing refers to the steps involved in transforming a high-level query (e.g., in SQL) into an efficient execution plan and then executing it to retrieve the required results. In a **Distributed DBMS**, query processing becomes more complex due to the distribution of data across multiple sites.

Purpose and Objectives of Query Processing

- **Correctness:** The result should be the same as if the query were executed on a centralized system.
- **Efficiency:** Reduce response time, communication cost, and local processing cost.
- **Resource Optimization:** Efficiently use CPU, memory, and network bandwidth.
- **Scalability:** Performance should not degrade significantly as the system grows.
- **Parallelism:** Exploit parallel execution to speed up processing.

Characteristics of Query Processors

A **Query Processor** is a component of a DBMS responsible for interpreting and executing database queries. In a DDBMS, its key characteristics are:

- **Data Location Awareness:** It must know the location of data fragments across different sites.
- **Communication Management:** Handles inter-site communication to gather data.
- **Decomposition:** Breaks global queries into sub-queries suitable for local sites.
- **Optimization Capability:** Optimizes the query execution considering data distribution, replication, and network cost.
- **Heterogeneity Handling:** May need to work with different DBMSs or schemas at various sites.
- **Fault Tolerance:** Manages failures of sites or links during processing.

Layers of Query Processing

The process of query execution in a DDBMS can be divided into several layers:

a. Query Decomposition

- Translates the high-level query into an internal form.
- Checks for syntactic and semantic correctness.
- Converts query into relational algebra or calculus expressions.

b. Data Localization

- Identifies which fragments of the distributed data are relevant.
- Rewrites the query using fragment definitions.
- Resolves fragment locations (horizontal, vertical, derived fragmentation).

c. Global Optimization

- Considers various execution strategies across sites.
- Estimates costs (I/O, CPU, communication).
- Chooses an optimal plan based on cost models.

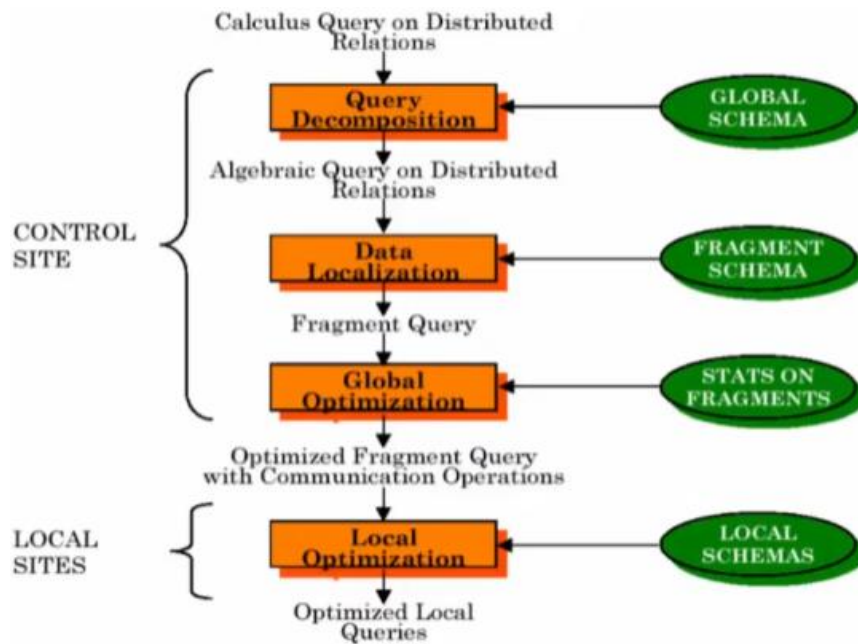
d. Local Optimization

- Optimizes subqueries locally at each site using local DBMS capabilities.
- Applies local indexes, join orders, etc.

e. Execution

- Executes the plan in a coordinated manner.
- Collects intermediate results.
- Combines and presents the final output to the user.

Layer	Input	Process (Work Done)	Output
1. Query Decomposition	- High-level user query (e.g., SQL) - Global schema and integrity constraints	- Parses and checks for syntax & semantics - Translates query into relational algebra - Verifies correctness	- Canonical relational algebra query
2. Data Localization	- Canonical relational algebra query - Fragmentation schema - Fragment location info	- Rewrites query using fragment definitions - Maps subqueries to data fragments at sites	- Fragmented query with site-specific subqueries
3. Global Query Optimization	- Fragmented query - Data distribution metadata - Cost models (CPU, I/O, communication)	- Generates alternative execution strategies - Estimates costs - Chooses most efficient global plan	- Optimized global query execution plan
4. Local Optimization	- Subqueries for each site - Local DB schema and stats	- Applies local DBMS optimization (e.g., join ordering, index usage) - Refines physical query plans	- Optimized subquery execution plans for each site
5. Query Execution	- Local and global execution plans - Actual distributed data	- Executes subqueries at respective sites - Manages inter-site communication - Merges intermediate results	- Final result to the user



Challenges in Distributed Environments

Query processing in distributed databases faces several unique challenges:

- **Data Fragmentation and Replication:** Managing and locating multiple fragments or replicas.
- **Network Latency and Bandwidth:** Communication costs can dominate execution time.
- **Site Autonomy:** Sites may be independent and may not cooperate fully.
- **Heterogeneity:** Different DBMSs, data models, and query languages.
- **Dynamic Load Balancing:** Need to distribute workload efficiently across nodes.
- **Fault Tolerance:** Ensuring query completion despite site/link failures.
- **Security and Privacy:** Ensuring safe data exchange across potentially insecure links.

1. Query decomposition

Query decomposition is the first stage of query processing where a high-level calculus query (e.g., SQL) is transformed into an intermediate, simplified, and logically equivalent form—usually relational algebra (with algebraic operations like select, join, rename etc).

It ensures that the query is syntactically and semantically correct before any distributed optimization or execution occurs.

Why Query Decomposition?

- Improves performance by simplifying and optimizing before distribution.
- Helps in **localizing** subqueries to specific sites.
- Enables **parallelism** and **site autonomy** in execution.

Steps of Query Decomposition

1. Normalization

- Transforms a complex query into a simpler, standardized form.
- E.g., converting nested queries into flat relational algebra expressions.
- Goal: Simplify the query structure.

2. Analysis

- Checks the **query's syntactic and semantic correctness**.
- Ensures that attributes exist, types match, etc.
- Also checks for **query equivalence** to maintain correctness.

3. Simplification / Elimination of Redundancy

- Removes unnecessary operations like redundant joins, selections, or projections.
- Example: $\text{SELECT } * \text{ WHERE age} > 25 \text{ AND age} > 20 \rightarrow \text{eliminate age} > 20$.

4. Restructuring / Canonical Transformation

- Converts the query into an **equivalent but more efficient** form.
- May involve using **heuristic rules** like pushing selections down or combining joins.

A. Normalization

① It is the goal of normalization to transform the query to a normal form to facilitate processing.

② It is the 1st stage of Query decomposition, where the SQL query is converted to relational algebra exp.

③ Simplifies nested queries, subqueries and complex expr into basic operations (like selection, projection, join etc).

④ There are 2 normal forms it is converted to —
→ Conjunctive NF $[(p_a \vee p_b \dots p_n) \wedge \dots \wedge (p_x \vee p_y \dots p_m)]$
→ Disjunctive NF $[(p_a \wedge p_b \dots p_n) \vee \dots \vee (p_x \wedge p_y \dots p_m)]$

Example

Select Ename From EMP, ASG

Where EMP.Eno = ASG.Eno AND ASG.Pro = "P1"
AND DUR = 12 OR DUR = 24

Ans) CNF

$\text{EMP.Eno} = \text{ASG.Eno} \wedge \text{ASG.Pro} = "P1" \wedge (\text{DUR} = 12 \vee \text{DUR} = 24)$

DNF

$(\text{EMP.Eno} = \text{ASG.Eno} \wedge \text{ASG.Pro} = "P1" \wedge \text{DUR} = 12) \vee$
 $(\text{EMP.Eno} = \text{ASG.Eno} \wedge \text{ASG.Pro} = "P1" \wedge \text{DUR} = 24)$

Any of the above may be selected but, DNF contain redundant subexpressions. We select CNF.

$\therefore \Pi_{\text{Ename}} (\sigma_{\text{EMP.Eno} = \text{ASG.Eno} \wedge \text{ASG.Pro} = "P1" \wedge (\text{DUR} = 12 \vee \text{DUR} = 24)} (\text{EMP} \times \text{ASG}))$

B. Analysis

Identify and reject type incorrect and semantically incorrect queries

2. **Type Incorrect Queries:** A type incorrect query refers to a query where data types are used improperly or inconsistently, making the operation invalid or meaningless. Occurs When:

- Operations are applied to incompatible data types.
- Functions expecting specific types are given wrong ones.

Examples:

- a. Comparing incompatible types: salary is numeric, but 'high' is a string → type mismatch.

```
SELECT * FROM Employee WHERE salary = 'high';
```

- b. Performing arithmetic on non-numeric fields: name is a string; addition with a number is type incorrect.

```
SELECT name + 100 FROM Student;
```

- c. Using aggregate functions on wrong types: AVG() expects numeric values, but name is a string.

```
SELECT AVG(name) FROM Student;
```

3. **Semantically Incorrect Queries:** A semantically incorrect query is one that is syntactically and type-wise valid, but does not make logical sense in the context of the schema or database semantics.

Occurs When:

- The query references nonexistent attributes or relations.
- Operations are attempted on invalid structures.
- Query logic contradicts the schema's meaning or constraints.

Examples:

- a. Referencing nonexistent attribute: roll_no is not part of the Course table → semantically invalid.

```
SELECT roll_no FROM Course;
```

- b. Invalid join condition: This is syntactically correct but logically makes no sense, unless name and title are meant to be equal, which is unlikely.

```
SELECT * FROM Student, Course WHERE Student.name = Course.title;
```

- c. Using a relation that doesn't exist: If Marks is not a defined table, the query is semantically incorrect.

```
SELECT * FROM Marks;
```

- d. Checking access control (if the user lacks permission): Table exists, but user doesn't have access → semantically invalid from a system perspective.

```
SELECT * FROM Salary;
```

We use different data structures to detect the semantically incorrect queries:

1. Connection Graph (or Query Graph)

- **Nodes** represent **relations (tables)**.
- **Edges** represent **join conditions** between the relations if both connected nodes represent a operand relation, else it is **projection**.
- To represent the **structure of a query**, especially **relationships among relations (tables)** involved in a query.
- Used to detect **missing joins, invalid references, or disconnected relations**, which indicate **semantic errors**.

2. Join Graph

- It is a subgraph of the query graph that represents only the **join conditions** among relations.
- If the query graph is connected the query is semantically correct else not.
- Nodes:** relations involved in the join
- Edges:** join predicates, typically equi-joins on foreign key relationships

Example of analysis

Consider the following 2 queries. Using query graph and join graph state if they are semantically correct.

QUERY 1

```
SELECT ENAME, RESP
FROM EMP, ASG, PROJ
WHERE EMP.ENO = ASG.ENO
AND ASG.PNO = PROJ.PNO
AND PNAME = 'CAD/CAM'
AND DUR >= 36
AND TITLE = 'Programmer';
```

QUERY 2

```
SELECT ENAME, RESP
FROM EMP, ASG, PROJ
WHERE EMP.ENO = ASG.ENO
AND PNAME = 'CAD/CAM'
AND DUR >= 36
AND TITLE = 'Programmer';
```

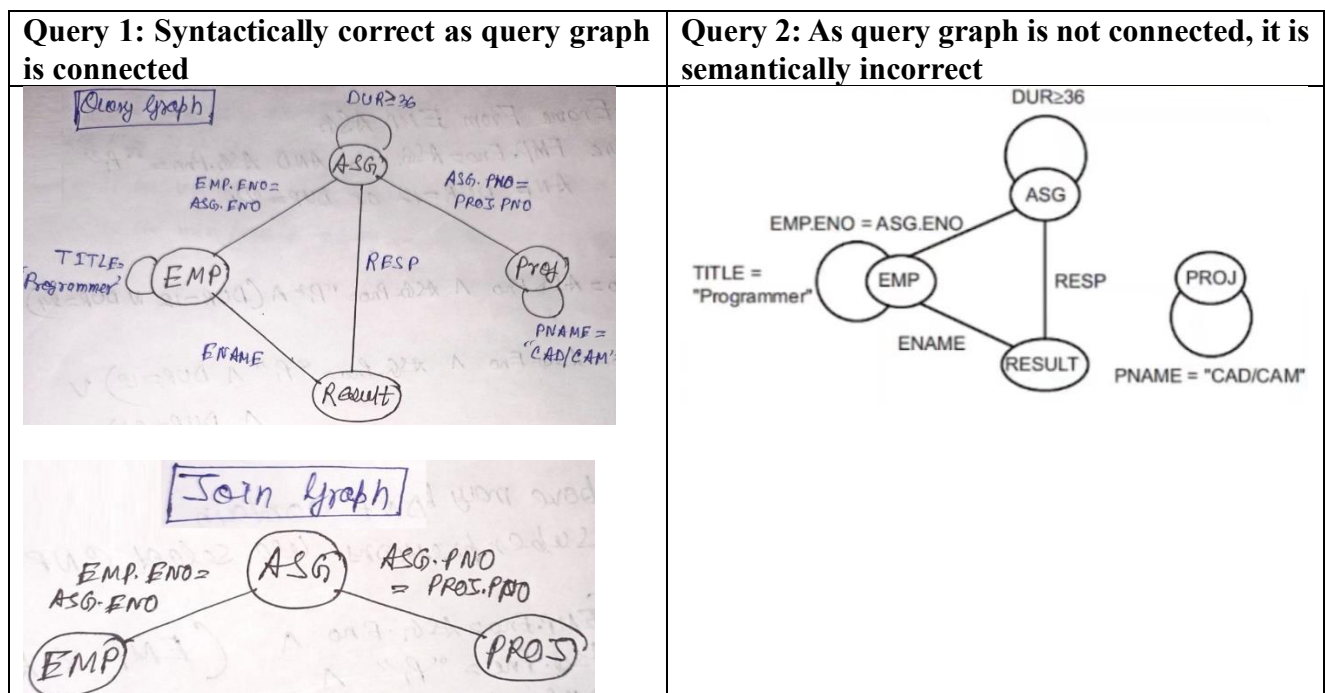
ENO	ENAME	TITLE
E1	J. Doe	Elect. Eng
E2	M. Smith	Syst. Anal.
E3	A. Lee	Mech. Eng.
E4	J. Miller	Programmer
E5	B. Casey	Syst. Anal.
E6	L. Chu	Elect. Eng.
E7	R. Davis	Mech. Eng.
E8	J. Jones	Syst. Anal.

ENO	PNO	RESP	DUR
E1	P1	Manager	12
E2	P1	Analyst	24
E2	P2	Analyst	6
E3	P3	Consultant	10
E3	P4	Engineer	48
E4	P2	Programmer	18
E5	P2	Manager	24
E6	P4	Manager	48
E7	P3	Engineer	36
E8	P3	Manager	40

PNO	PNAME	BUDGET
P1	Instrumentation	150000
P2	Database Develop.	135000
P3	CAD/CAM	250000
P4	Maintenance	310000

TITLE	SAL
Elect. Eng.	40000
Syst. Anal.	34000
Mech. Eng.	27000
Programmer	24000

Answer



C. Elimination of Redundancy

Elimination of redundancy is the step where the system removes unnecessary or logically repeated parts of a query to make it simpler, more efficient, and semantically correct.

Example of redundancy

```
SELECT TITLE FROM EMP
WHERE EMP.ENAME = "J. Doe"
      OR (NOT(EMP.TITLE = "Programmer")
          AND (EMP.TITLE = "Elect. Eng." OR EMP.TITLE = "Programmer")
          AND NOT(EMP.TITLE = "Elect. Eng."))
```

Answer

The above query can be simplified by considering this method

- p1: TITLE = "Programmer"
- p2: TITLE = "Elect. Eng."
- p3: ENAME = "J. Doe"

Handwritten logical simplification of the query:

$$\begin{aligned} & p_3 \vee (\neg p_1 \wedge (p_2 \vee p_1) \wedge \neg p_2) \\ &= p_3 \vee ((\neg p_1 \wedge p_2) \vee (\neg p_1 \wedge p_1)) \wedge \neg p_2 \\ &= p_3 \vee ((\neg p_1 \wedge p_2 \wedge \neg p_2) \vee (\neg p_1 \wedge p_1 \wedge \neg p_2)) \\ &= p_3 \vee \text{false} \vee \text{false} \\ &= p_3 \\ \therefore \text{Simplified query: Select Title from EMP} \\ &\quad \text{where Ename = "J.Doe"} \end{aligned}$$

D. Rewriting

- **Rewriting** is the **final step** of query decomposition in Distributed DBMS.
- In this step, the original high-level query (usually written in relational calculus or SQL) is **rewritten into relational algebra**.
- This algebraic expression is then represented **graphically using an operator tree**.
- **Operator tree**: is a hierarchical representation where:
 - **Leaves** represent base relations (tables).
 - **Intermediate nodes** represent algebraic operations like **selection (σ)**, **projection (π)**, **join ($\bowtie$)**, etc.
 - The **root** represents the final output of the query.
- Multiple **equivalent relational algebra queries** can be derived from the same relational calculus (or SQL) query.
- However, not all of them are equally efficient, some **algebraic expressions are better** in terms of:
 - **Query cost**
 - **Communication overhead**
 - **Local processing time**

Query: Find the names of employees other than J. Doe who worked on CAD/CAM proj for either 12 or 24 yrs.

SQL query

Select Ename
From EMP, ASG, PROJ
Where EMP.Enid=ASG.Enid
and ASG.Proj=PROJ.Proj
and ~~EMP~~.Ename $\neq$ 'J. Doe'
and ~~PROJ~~.Pname = 'CAD/CAM'
and (DUR=12 $\vee$ DUR=24)

Diagram illustrating the query execution flow:

```
graph BT
    PROJ --> Pname["σ Pname = 'CAD/CAM'"]
    Pname --> Dur["σ DUR = 12 ∨ DUR = 24"]
    Dur --> Ename["π Ename"]
```

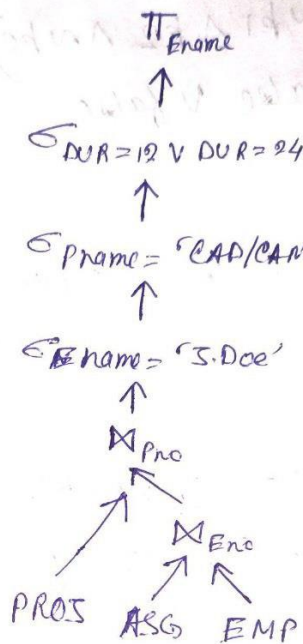
The diagram shows the execution flow starting from the PROJ table, filtering for Pname = 'CAD/CAM', then filtering for DUR = 12 or DUR = 24, and finally projecting the Ename column.

Select Ename

where EMP. $E_{no} = AS6, E_{no}$

and ~~Emp~~ $Emp \Leftarrow 'J. Doe'$

and ($DUR = 12$ v $DUR = 24$)



2. Data Localization

-
- ```

graph TD
 A[Algebraic Query on Distributed Relations] --> B[Data Localization]
 C([FRAGMENT SCHEMA]) --> B
 B --> D[Fragment Query]

```

## Translating a Global Query

- In a DDBMS, a **global relation** (e.g., Student) may be **fragmented** and stored at different sites.
- Additionally, fragments might be **replicated** for efficiency and fault tolerance.
- A **user** or application issues a query on the **global schema**, without knowledge of where data is physically stored.

## Example: decomposition of global query into fragments

Global Query: `SELECT * FROM Student;`

If the `Student` relation is horizontally fragmented into multiple parts:

- `Student1`
- `Student2`
- ...
- `StudentN`

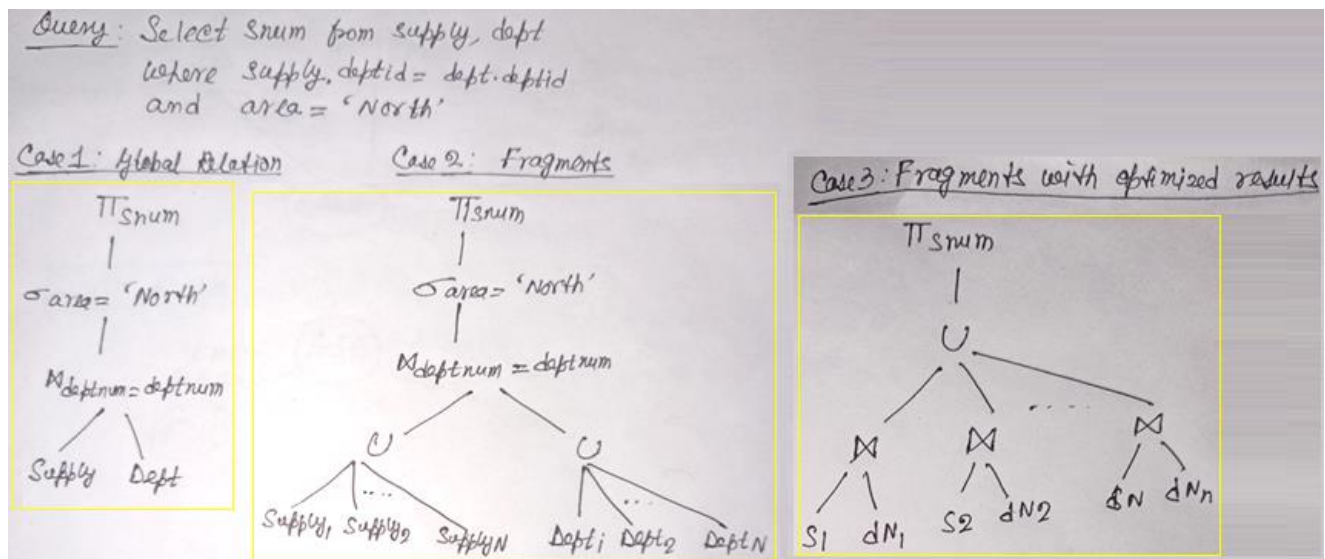
Then, the global query is decomposed into the following fragment queries:

```
SELECT * FROM Student1;
SELECT * FROM Student2;
...
SELECT * FROM StudentN;
```

To execute these fragment queries, the system must know:

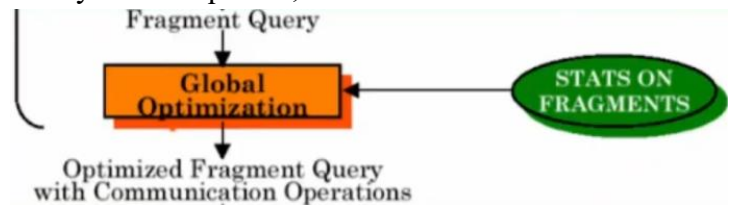
- How the `Student` relation was **fragmented**
- Whether any fragments are **replicated**
- Where each fragment or its replica is **allocated** (i.e., stored)

## Example: Query tree representation



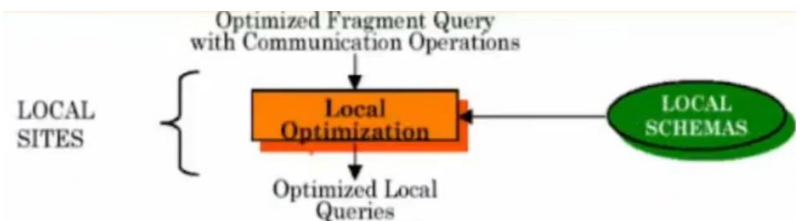
### 3. Global Query Optimization

- **Input:** Algebraic query on fragments (after data localization).
- **Objective:** Determine an execution strategy that minimizes overall cost (close to optimal).
- **Key Concepts:**
  - Permutation on queries leads to many similar queries, which are ordered based on cost functions, which includes:
    - Time units
    - Disk space
    - I/O operations
    - CPU usage
    - Communication cost (e.g., limited bandwidth)
  - The goal is to minimize total execution cost.
- **Factors Considered:**
  - Fragment allocation and replication
  - Fragment statistics (e.g., tuple count, selectivity)
- **Static Optimization:** cost is predicted statistically before execution, with techniques like
  - Join ordering
  - Use of semijoins to reduce data transfer
- **Output:** Optimized algebraic query with **communication operators** and **fragment references**.



### 4. Local Optimization

- **Performed at:** Each site that stores relevant fragments.
- **Purpose:** Optimize and execute local subqueries using the site's local schema and resources.
- **Steps:**
  - Choose appropriate algorithms for relational operators (e.g., join methods).
  - Optimize query execution plan locally.
- **Result:** Local results are computed and sent back to the query origin site for final assembly.



## Query Processing vs Query Optimization

| Aspect           | Query Processing                                                               | Query Optimization                                                 |
|------------------|--------------------------------------------------------------------------------|--------------------------------------------------------------------|
| Definition       | Entire process of executing a query from parsing to result                     | Subprocess focused on improving query efficiency                   |
| Scope            | Broad – includes decomposition, localization, optimization, execution          | Narrow – only includes finding the best execution strategy         |
| Steps Involved   | Query decomposition, data localization, global & local optimization, execution | Global and local optimization                                      |
| Purpose          | Get correct results from the DB                                                | Reduce cost, time, and resources used during execution             |
| Focus            | Execution pipeline                                                             | Efficiency of execution strategy                                   |
| Timing           | Runs from start to end of query life cycle                                     | Happens before actual execution                                    |
| Techniques Used  | Parsing, rewriting, access planning, execution                                 | Cost-based search, heuristic rules, transformation-based rewriting |
| System Component | Handled by the query processor module                                          | Handled by the query optimizer module                              |

# CH 3.1: DISTRIBUTED QUERY OPTIMIZATION

## Contents

|                                                                    |    |
|--------------------------------------------------------------------|----|
| Semi Join ( $\bowtie$ ).....                                       | 1  |
| Distributed Query Optimization .....                               | 1  |
| Direct Join Ordering.....                                          | 2  |
| Semi Join Based Algorithm .....                                    | 3  |
| System R Join Algorithm (Nested Loop Join with Optimizations)..... | 4  |
| R* Algorithm.....                                                  | 5  |
| 2 methods in R* for inter-site data transfer .....                 | 6  |
| Join Processing and Cost Formulas in R* Algorithm.....             | 6  |
| Hill Climbing.....                                                 | 9  |
| SDD-1 Algorithm .....                                              | 9  |
| Distributed Cost Model.....                                        | 9  |
| Factors Governing Query Optimization .....                         | 10 |
| Centralized and Decentralized Query Optimization .....             | 11 |

## Semi Join ( $\bowtie$ )

A semi join between two tables A and B returns only the rows from A that have a matching row in B, but does not return B's columns. It is useful in distributed environments to reduce data transfer between sites. Example:

**Student (at Site A)**

| Sid | Sname | Tid |
|-----|-------|-----|
| 1   | Ram   | T1  |
| 2   | Shyam | T2  |
| 3   | Sita  | T3  |
| 4   | Radha | T1  |

**Teacher (at Site B)**

| Tid | Tname  |
|-----|--------|
| T1  | Shiva  |
| T2  | Vishnu |

**Student  $\bowtie$  Teacher (at site A)**

| Sid | Sname | Tid |
|-----|-------|-----|
| 1   | Ram   | T1  |
| 2   | Shyam | T3  |
| 4   | Radha | T1  |

## Distributed Query Optimization

In DDBMS, join operations across different sites can cause significant communication costs, making join ordering crucial.

### Why Join Ordering Matters

- **In centralized DBMS:** Join order affects performance (e.g., System R optimizer).
- **In distributed DBMS:** Data transfer cost is added, making efficient join planning even more critical.

### Two Main Approaches

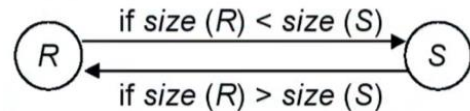
#### i. Direct Join Ordering

- Optimize the order in which joins are executed.
- Systems: **INGRES and Distributed INGRES, System R and System R\***

- **Strategy:**
  - Move the smaller relation to the site of the larger one.
  - Estimate sizes of relations (R and S) to minimize data movement.
- ii. **Join Replacement using Semi joins**
  - Replace full joins with **semi join sequences** to reduce data transfer.
  - Systems: **Hill Climbing, SDD-1**
  - Especially effective when:
    - Relations are large
    - Selectivity is high (few matches)

## Direct Join Ordering

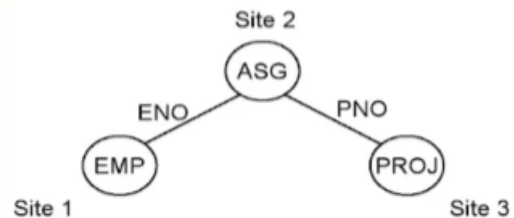
- If relation **R** is at **Site A**, and relation **S** is at **Site B**:
  - Move the **smaller** of R or S to the other site.
  - Perform the join locally after transfer.
- Direct join ordering of queries involving more than 2 relation is substantially more complex.



### Example:

This query can be evaluated in the following ways:

- 1) EMP → Site 2
- 2) Site 2: EMP' = EMP ⋈ ASG
- 3) EMP' → Site 3
- 4) Site 3: EMP' ⋈ PROJ



- 1) PROJ → Site 2
- 2) Site 2: PROJ' = PROJ ⋈ ASG
- 3) PROJ' → Site 1
- 4) Site 1: PROJ' ⋈ EMP

- 1) ASG → Site 1
- 2) Site 1: EMP' = EMP ⋈ ASG
- 3) EMP' → Site 3
- 4) Site 3: EMP' ⋈ PROJ

- 1) ASG → Site 3
- 2) Site 3: PROJ' = PROJ ⋈ ASG
- 3) PROJ' → Site 1
- 4) Site 1: PROJ' ⋈ EMP

- 1) EMP → Site 2
- 2) PROJ → Site 2
- 3) Site 3: EMP ⋈ PROJ ⋈ ASG

To select a plan from above, a lot of information are needed:

- Size (EMP)
- Size (ASG)
- Size (PROJ)
- Size (EMP ⋈ ASG)
- Size (ASG ⋈ PROJ)

Use **direct join** only if:

- **R is already small**
- Simplicity is prioritized over communication cost

## Semi Join Based Algorithm

**Assuming:** The join of 2 relation R and S over attributes R stored at sites 1 and 2 respectively

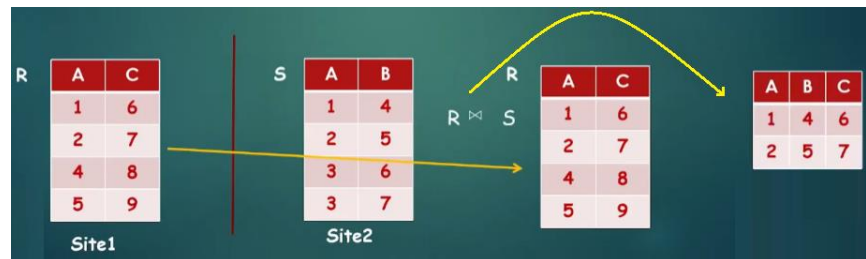
**Cost Analysis:**

- Direct Join ordering:  $R \bowtie_A S$
- Semi join based algorithm:  $(R \bowtie_A S) \bowtie S$

(assuming size (R) < size (S))

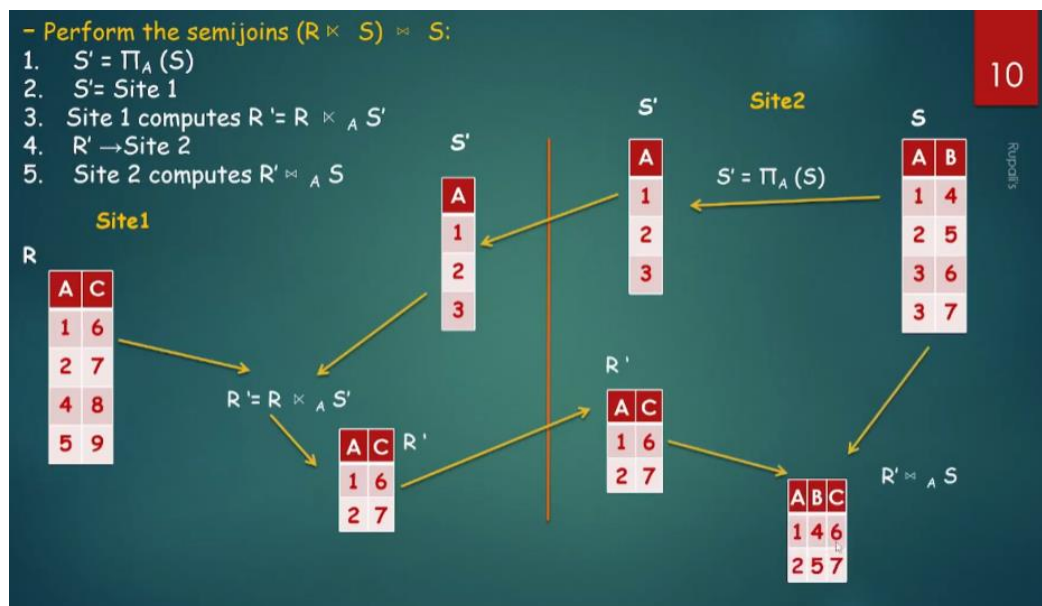
a) Perform the join  $R \bowtie_A S$   
(Direct join ordering)

- $R \rightarrow$  site 2
- Site 2 computes  $R \bowtie S$



b) Perform the semijoins  $(R \bowtie S) \bowtie_A S$

- $S' = \pi_A(S)$
- $S' \rightarrow$  Site 1
- Site 1 computes  $R' = R \bowtie_A S'$
- $R' \rightarrow$  Site 2
- Site 2 computes  $R' \bowtie_A S$



- We can conclude that semi join is better if  $\text{sizeof}(\pi_A(S)) + \text{size}(R \bowtie S) < \text{size}(R)$
- The semi join approach is better if it acts as a sufficient reducer (few tuples participate in join)
- **Semijoin-based strategy (b) is better** in most distributed environments, especially when:
  - **R is large**
  - **Join selectivity is low**
  - **We want to reduce network traffic**
- Use **direct join (a)** only if:
  - **R is already small**
  - Simplicity is prioritized over communication cost

## System R Join Algorithm (Nested Loop Join with Optimizations)

The **System R** algorithm developed by IBM was a pioneering RDBMS that introduced a **cost-based optimizer** and practical join strategies. Its join algorithm is based on the **block nested loop join**, optimized using access paths (indexes, sort orders, etc.).

- **Cost-based:** Selects the join plan with the lowest estimated cost.
- **Dynamic programming based**
- **Semi-joins:** Used to reduce unnecessary data transfer.
- **Data localization:** Tries to perform operations at data source sites.
- **Communication-efficient:** Minimizes inter-site data movement.
- **Statistics-driven:** Relies on metadata like table size and selectivity.
- **Distributed join support:** Handles hash, sort-merge, and nested-loop joins.
- **Site autonomy:** Works with independently managed database sites.
- **Scalable:** Efficient with increasing sites and data size.
- **Multi-query optimization:** Shares results across multiple queries to avoid redundancy.
- **Adaptive execution:** Can adjust execution plan based on run-time info.

### Join Strategy in System R

System R uses a dynamic programming approach and supports two main join algorithms:

1. Nested-Loop Join (NLJ)
2. Sort-Merge Join (SMJ)

It estimates the cost of each plan using statistics, and chooses the least-cost plan.

### Assumptions:

- Relations **R** and **S** need to be joined on a condition (e.g.,  $R.A = S.B$ )
- Multiple access paths (sequential scan, indexed access, etc.) may be available
- **Smaller relation is outer**

### Example:

Names of employees working on CAD/CAM project

PROJ  $\bowtie$  PNO ASG  $\bowtie$  PNO EMP

| EMP |           |             | ASG |     |            |     |
|-----|-----------|-------------|-----|-----|------------|-----|
| ENO | ENAME     | TITLE       | ENO | PNO | RESP       | DUR |
| E1  | J. Doe    | Elect. Eng  | E1  | P1  | Manager    | 12  |
| E2  | M. Smith  | Syst. Anal. | E2  | P1  | Analyst    | 24  |
| E3  | A. Lee    | Mech. Eng.  | E2  | P2  | Analyst    | 6   |
| E4  | J. Miller | Programmer  | E3  | P3  | Consultant | 10  |
| E5  | B. Casey  | Syst. Anal. | E3  | P4  | Engineer   | 48  |
| E6  | L. Chu    | Elect. Eng. | E4  | P2  | Programmer | 18  |
| E7  | R. Davis  | Mech. Eng.  | E5  | P2  | Manager    | 24  |
| E8  | J. Jones  | Syst. Anal. | E6  | P4  | Manager    | 48  |
|     |           |             | E7  | P3  | Engineer   | 36  |
|     |           |             | E8  | P3  | Manager    | 40  |

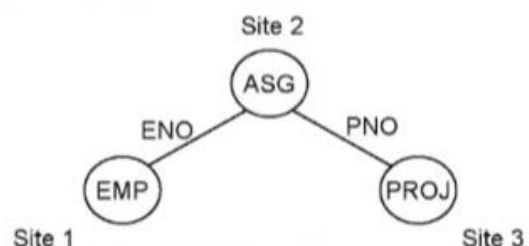
| PROJ |                   |        | PAY         |       |
|------|-------------------|--------|-------------|-------|
| PNO  | PNAME             | BUDGET | TITLE       | SAL   |
| P1   | Instrumentation   | 150000 | Elect. Eng. | 40000 |
| P2   | Database Develop. | 135000 | Syst. Anal. | 34000 |
| P3   | CAD/CAM           | 250000 | Mech. Eng.  | 27000 |
| P4   | Maintenance       | 310000 | Programmer  | 24000 |

### Answer

#### Indexes:

- EMP has an index on ENO
- ASG has an index on PNO
- PROJ has an index on PNO and an index on PNAME

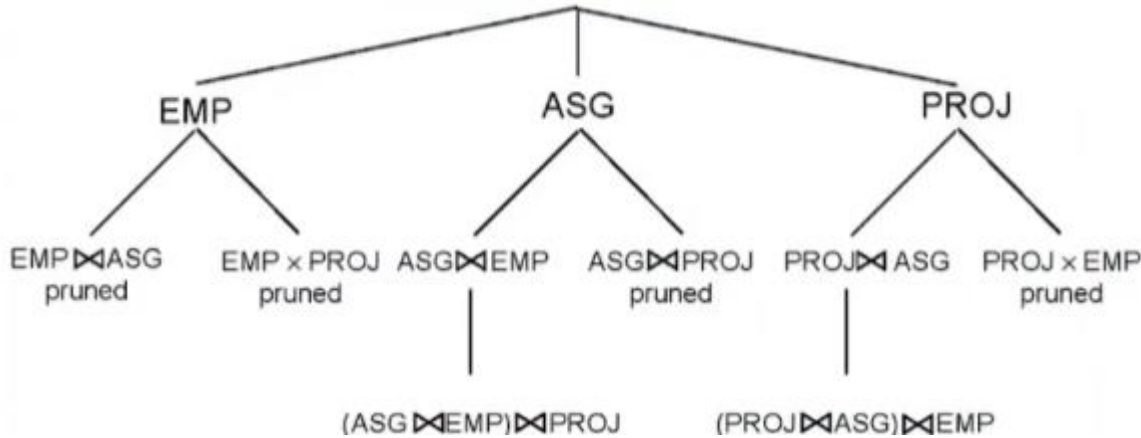
#### Join Graph:



### Step 1 - Select the best single-relation access paths

- EMP: sequential scan (because there is no selection condition on EMP)
- ASG: sequential scan (same)
- PROJ: index on PNAME (because there is a selection on PROJ based on PNAME=CAD/CAM)

### Step 2- Select the best join ordering for each relation



- The operations marked pruned are dynamically eliminated.
- (EMP × PROJ) and (PROJ × EMP) are pruned as cartesian products have no common attribute.
- We assume that (EMP ⋈ ASG) and (ASG ⋈ PROJ) have higher cost than (ASG ⋈ EMP) and (PROJ ⋈ ASG) respectively
- So, we get ((ASG ⋈ EMP) ⋈ PROJ) and ((PROJ ⋈ ASG) ⋈ EMP)
- Best total join order is ((PROJ ⋈ ASG) ⋈ EMP), as it uses the indexes best
  - Select PROJ using index on PNAME which is given in question
  - Join with ASG using index on PNO

## R\* Algorithm

The R\* algorithm is a query optimization algorithm which is an extension of the System R query — the distributed version of IBM's System R relational DBMS. It focuses on efficient query processing in distributed environments.

- Only entire relations are stored at each site (no fragmentation or replication).
- Query compilation happens in a distributed way but is controlled by the master site (where query starts).
- The **master site** decides:
  - Where each part of the query runs,
  - How joins are ordered,
  - How data moves between sites.
  - Join order, and
  - Minimizing data transfer between sites.
- **Local sites handle:**
  - **Accessing** data efficiently,
  - Doing joins on data stored locally.

### Assumptions of R\*:

- Distributed database with relations located at multiple sites.
- Communication cost between sites is significant.
- Goal is to minimize total cost (I/O + communication).

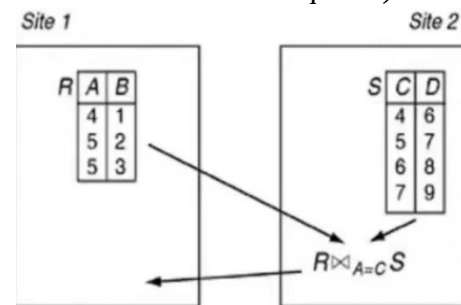
### Key Features of R\*:

- **Cost-based:** Selects plans that minimize total I/O, CPU, and communication cost.
- **Dynamic programming:** Builds optimal global plans from optimal subplans.
- **Semi-join strategy:** Uses semi-joins to reduce data transfer before full joins.
- **Data localization:** Prefers executing operations at data source sites.
- **Communication-efficient:** Minimizes inter-site data movement.
- **Statistics-driven:** Uses relation size, selectivity, and metadata for planning.
- **Distributed join support:** Handles hash, sort-merge, and nested-loop joins across sites.
- **Site autonomy:** Works with independently managed and autonomous sites.
- **Scalable:** Efficient even with many sites and large datasets.
- **Multi-query optimization:** Reuses intermediate results across queries.
- **Adaptive:** Can adjust execution based on run-time conditions (in advanced versions).

## 2 methods in R\* for inter-site data transfer

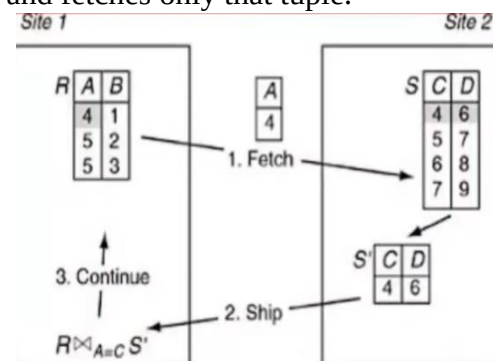
**1. Ship Whole:** Instead of fetching only required rows, transfer entire relations (tables) from remote sites to the site where the query will be executed (typically the site where the result is required).

- **Pros:**
  - Simple to implement.
  - Reduces the number of messages (just one large transfer).
  - Better if relations are small.
  - Larger data transfer
- **Cons:**
  - Very costly if the table is large.
  - Wastes bandwidth by transferring unnecessary data (i.e., rows not needed for the result).



**2. Fetch as Needed (Tuple Shipping):** Only the required rows are fetched from the remote sites as needed during query execution. The query is executed in a tuple-at-a-time manner. Whenever the local site needs a tuple from the remote site, it sends a request and fetches only that tuple.

- **Pros:**
  - Avoids unnecessary data transfer.
  - Efficient if only a few tuples are needed from the remote site.
  - Better if relations are large and selectivity is good.
- **Cons:**
  - High communication overhead due to many small messages.
  - Slower due to multiple network round-trips.



## Join Processing and Cost Formulas in R\* Algorithm

### Assumptions:

- R= external relation
- S=internal relation
- **card(R):** Number of tuples in external relation R
- **card(S):** Number of tuples in internal relation S
- **s:** Average number of S tuples matching one R tuple
- **length(A):** Size of the join attribute A
- **length(S):** Size of a tuple in S
- **CT(x):** Communication time of transferring x
- **LT(x):** Local processing (I/O + CPU) for x

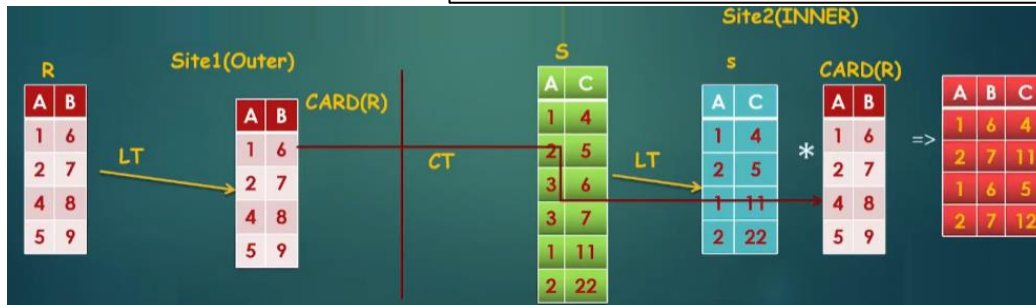
Average number of S tuples matching one R tuple =

$$s = \frac{\text{card}(S \bowtie_A R)}{\text{card}(R)}$$

### ◆ Strategy 1: Ship R(external) to S's(internal) site

External relation R is sent to the internal relation S's site.

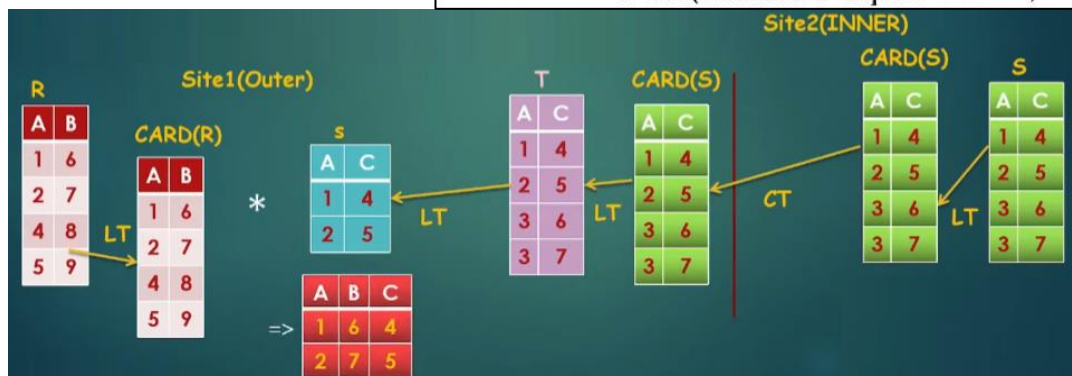
$$\begin{aligned} \text{Total\_cost} = & LT(\text{retrieve } \text{card}(R) \text{ tuples from } R) \\ & + CT(\text{size}(R)) \\ & + LT(\text{retrieve } s \text{ tuples from } S) * \text{card}(R) \end{aligned}$$



### ◆ Strategy 2: Ship S to R's site

Internal relation S is sent to R's site and stored temporarily.

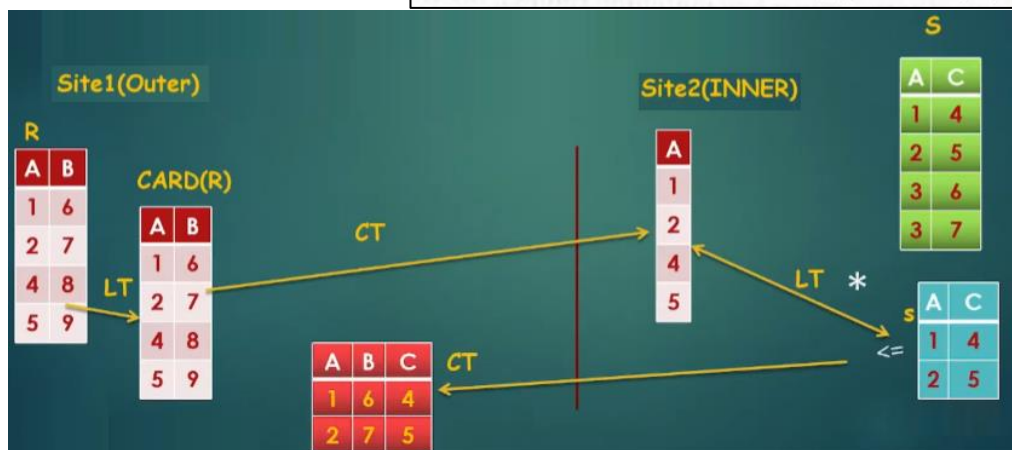
$$\begin{aligned} \text{Total\_cost} = & LT(\text{retrieve } \text{card}(S) \text{ tuples from } S) \\ & + CT(\text{size}(S)) \\ & + LT(\text{store } \text{card}(S) \text{ tuples in } T) \\ & + LT(\text{retrieve } \text{card}(R) \text{ tuples from } R) \\ & + LT(\text{retrieve } s \text{ tuples from } T) * \text{card}(R) \end{aligned}$$



### ◆ Strategy 3: Fetch S tuples as needed

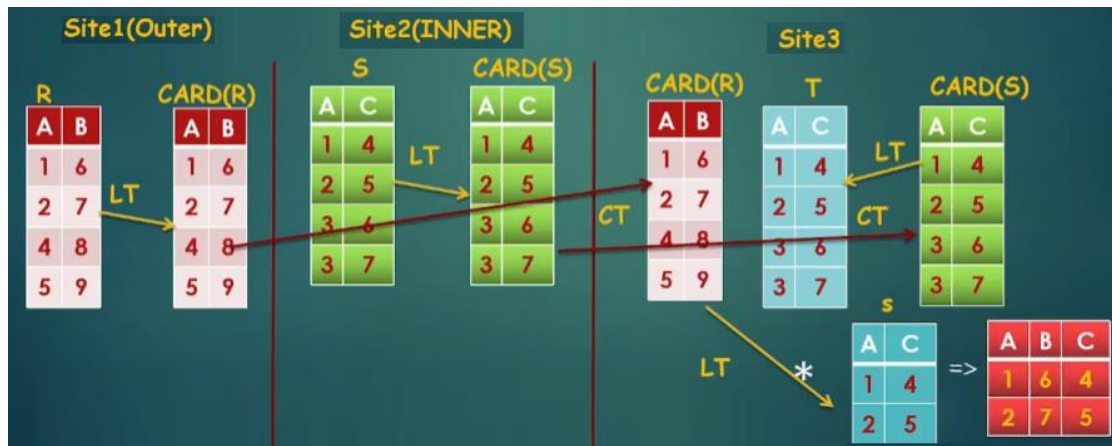
For each R tuple, fetch matching S tuples individually from S's site.

$$\begin{aligned} \text{Total\_cost} = & LT(\text{retrieve } \text{card}(R) \text{ tuples from } R) \\ & + CT(\text{length}(A)) * \text{card}(R) \\ & + LT(\text{retrieve } s \text{ tuples from } S) * \text{card}(R) \\ & + CT(s * \text{length}(S)) * \text{card}(R) \end{aligned}$$



◆ **Strategy 4: Move both to a third site**  
R and S are both shipped to a third site and joined there.

$$\begin{aligned}
 \text{Total\_cost} = & LT(\text{retrieve } \text{card}(S) \text{ tuples from } S) \\
 & + CT(\text{size}(S)) \\
 & + LT(\text{store } \text{card}(S) \text{ tuples in } T) \\
 & + LT(\text{retrieve } \text{card}(R) \text{ tuples from } R) \\
 & + CT(\text{size}(R)) \\
 & + LT(\text{retrieve } s \text{ tuples from } T) * \text{card}(R)
 \end{aligned}$$



| Strategy                | Comm. Cost                  | Local Processing    | Total Cost Trend    | Best When                                                                    | Worst When                         |
|-------------------------|-----------------------------|---------------------|---------------------|------------------------------------------------------------------------------|------------------------------------|
| Ship R to S             | Moderate (CT(size(R)))      | Moderate            | Usually lower       | R is small, S is large, and s is small                                       | R is large                         |
| Ship S to R             | High (CT(size(S)))          | High (store + join) | Often higher than 1 | S is very small and cheap to store                                           | S is large                         |
| Fetch S as needed       | Very high (many small CTs)  | Moderate            | Highest total cost  | s is very small and card(R) is low                                           | s or card(R) is large              |
| Move both to third site | Highest (move both R and S) | Highest             | Very high           | Rarely best; only if third site is fastest or local load balancing is needed | Usually impractical for large data |

### Example:

| PROJ(External)                                                         |                                                                               |        | ASG(Internal)                                                                      |     |      |     |
|------------------------------------------------------------------------|-------------------------------------------------------------------------------|--------|------------------------------------------------------------------------------------|-----|------|-----|
| PNO                                                                    | PNAME                                                                         | Budget | ENO                                                                                | PNO | RESP | DUR |
| <b>Strategy</b>                                                        | <b>When Preferred</b>                                                         |        | <b>Remarks</b>                                                                     |     |      |     |
| <b>Strategy 4</b><br>(Move both to third site)                         | Almost never unless third site is extremely fast or needed for load balancing |        | ✗ Highest cost – both relations transferred                                        |     |      |     |
| <b>Strategy 2</b><br>(Ship internal ASG to external PROJ)              | ✓ Best when PROJ is much larger than ASG                                      |        | ✓ Minimizes communication if ASG is small and local processing is not high         |     |      |     |
| <b>Strategy 1 &amp; 3</b><br>(Ship PROJ to ASG or fetch ASG as needed) | Used when Strategy 2 isn't optimal                                            |        | ⚖️ Local processing cost is similar for both; choose based on s and comm. overhead |     |      |     |

| Aspect               | System R                  | R*                                             |
|----------------------|---------------------------|------------------------------------------------|
| Environment          | Centralized DBMS          | Distributed DBMS                               |
| Optimization Goal    | Minimize I/O and CPU cost | Minimize I/O, CPU, and communication           |
| Data Location        | Single site               | Multiple sites                                 |
| Join Ordering        | Dynamic programming based | Dynamic programming with <b>site awareness</b> |
| Semijoin Usage       | Not used                  | Used to reduce data transfer                   |
| Access Path Choice   | Based on local statistics | Based on <b>site and network stats</b>         |
| Cost Estimation      | Local costs only          | <b>Local + transfer cost</b> considered        |
| Intermediate Results | Local storage             | Might be <b>transferred between sites</b>      |
| Communication Cost   | Ignored                   | <b>Explicitly minimized</b>                    |

## Hill Climbing

- **Type:** Heuristic (greedy) search method
- **Approach:** Starts with an initial query plan, then **iteratively applies transformations** to move to a **neighbor plan** with **lower estimated cost**.
- **Goal:** Reach a **locally optimal plan** (minimum cost)
- **Limitation:** May get stuck in **local minima** (not guaranteed to find global optimum)
- **Usage:** Fast and simple, used when exhaustive search (like dynamic programming) is too costly.

## SDD-1 Algorithm

- **Type:** Rule-based heuristic optimization algorithm
- **Used in:** SDD-1 system — a pioneering distributed database system
- **Approach:**
  - Converts high-level SQL query to an **algebraic representation**
  - Applies **semantic query transformations**
  - Optimizes **data localization, join ordering, and data transfer**
- **Key Features:**
  - Emphasizes **data localization** (execute as much as possible at the data site)
  - Supports **semi-joins** to reduce data transfer
- **Goal:** Minimize **total cost** = local processing + communication cost

## Distributed Cost Model

- The cost of a distributed execution strategy can be expressed with respect to either the total time or the response time.
- Total time is sum of all time components.

$$\text{Total\_Time} = T_{\text{CPU}} \times \# \text{insts} + T_{\text{I/O}} \times \# \text{I/Os} + T_{\text{MSG}} \times \# \text{msgs} + T_{\text{TR}} \times \# \text{bytes}$$

- Response time is the elapsed time from the initiation to the completion of the query

$$\text{Response\_Time} = \max (T_{\text{CPU}} \times \# \text{insts}, T_{\text{I/O}} \times \# \text{I/Os}, T_{\text{MSG}} \times \# \text{msgs}, T_{\text{TR}} \times \# \text{bytes})$$

- RT based on communication time

$$\text{Communication\_Time} = T_{\text{MSG}} \times \# \text{msgs} + T_{\text{TR}} \times \# \text{bytes}$$

$$\text{Response\_Time} = \max(T_{\text{MSG}} + T_{\text{TR}} \times x, T_{\text{MSG}} + T_{\text{TR}} \times y)$$

$T_{\text{CPU}}$  = time of a CPU instruction

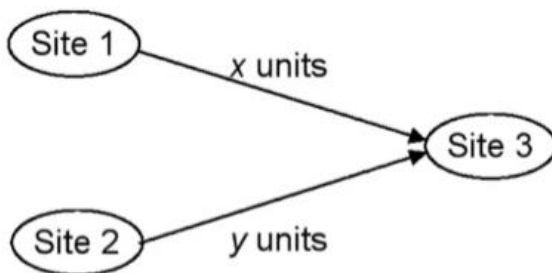
$T_{\text{I/O}}$  = time of a disk I/O

The communication time is depicted by the two last components.

$T_{\text{MSG}}$  = fixed time of initiating and receiving a message

$T_{\text{TR}}$  = time it takes to transmit a data unit from one site to another

**Example:**



find total time and response time

**Answer:**

We see no CPU time or IO time is mentioned here, so,

$$\begin{aligned} \text{Total time} &= \# \text{msg} * T_{\text{msg}} + T_{\text{tr}} * \# \text{bytes} \\ &= 2T_{\text{msg}} + T_{\text{tr}}(x+y) \end{aligned}$$

$$\text{Communication time for Site1 to Site3} = 1 * T_{\text{msg}} + T_{\text{tr}} * x$$

$$\text{Communication time for Site2 to Site3} = 1 * T_{\text{msg}} + T_{\text{tr}} * y$$

$$\text{Response time} = \max(T_{\text{msg}} + T_{\text{tr}} * x, T_{\text{msg}} + T_{\text{tr}} * y)$$

## Factors Governing Query Optimization

1. **Data distribution** – Location and fragmentation of data across sites
2. **Network latency** – Delay in data transmission between sites
3. **Communication cost** – Message startup and data transfer time
4. **Local processing cost** – CPU and disk I/O time at individual sites
5. **Query structure** – Complexity, joins, subqueries, and nesting
6. **Site autonomy** – Degree of control each site has over its data
7. **Data replication** – Multiple copies of data affecting choice of source
8. **System load** – Current workload and resource availability at sites
9. **Access methods** – Availability of indexes or access paths
10. **Intermediate result size** – Volume of data generated during processing
11. **Join ordering** – Sequence in which relations are joined
12. **Parallelism** – Potential to execute parts of the query concurrently

# Centralized and Decentralized Query Optimization

## 1. Centralized Query Optimization

- a. Done at a single central site
- b. Has global knowledge of data distribution and system state
- c. Generates one global execution plan
- d. Simpler control, but may be a bottleneck
- e. Suitable for tightly coupled systems
- f. Algorithms: INGRES algorithm, System R algorithm

## 2. Decentralized Query Optimization

- a. Done locally at each site
- b. Each site optimizes based on partial/local knowledge
- c. Results are combined for overall execution
- d. Scalable and avoids central bottlenecks
- e. Suitable for loosely coupled or federated systems
- f. Algorithm: R\* algorithm, SDD-1 algorithm, distributed INGRES algorithm

# CH 3.2: TRANSACTIONS

## Contents

|                                                 |   |
|-------------------------------------------------|---|
| Types of Transactions .....                     | 1 |
| A. Based on Transaction Modeling Approach.....  | 1 |
| B. Based on Execution Scope and Structure ..... | 2 |

## Types of Transactions

### A. Based on Transaction Modeling Approach

#### 1. General Model

The most flexible transaction model.

- A transaction can consist of **multiple sub-transactions**.
- Each sub-transaction can run at **different sites**.
- Sub-transactions can **spawn further sub-transactions**.
- **Use Case:** Highly complex distributed applications with dynamic behavior.

#### 2. Two-Step Model

- A restricted version of the general model with **two levels**.
- One **root transaction** that initiates multiple **independent sub-transactions**.
- Sub-transactions **do not create further transactions**.
- **Use Case:** Easier to manage, suitable for moderately complex systems.

#### 3. Restricted Model

- A simpler model where all operations are at **one level**.
- The transaction is split into sub-transactions **without nesting**.
- Sub-transactions must **run sequentially** or follow a strict order.
- **Use Case:** Simple workflow systems with clear task ordering.

#### 4. Restricted Two-Step Model

- Combines limitations of the **Restricted** and **Two-Step** models.
- Only **two levels** allowed (main + sub-transactions).
- Sub-transactions are **restricted**: no further sub-task creation, and limited communication.
- **Use Case:** Environments needing **predictability and control**, like workflows or sequential processing.

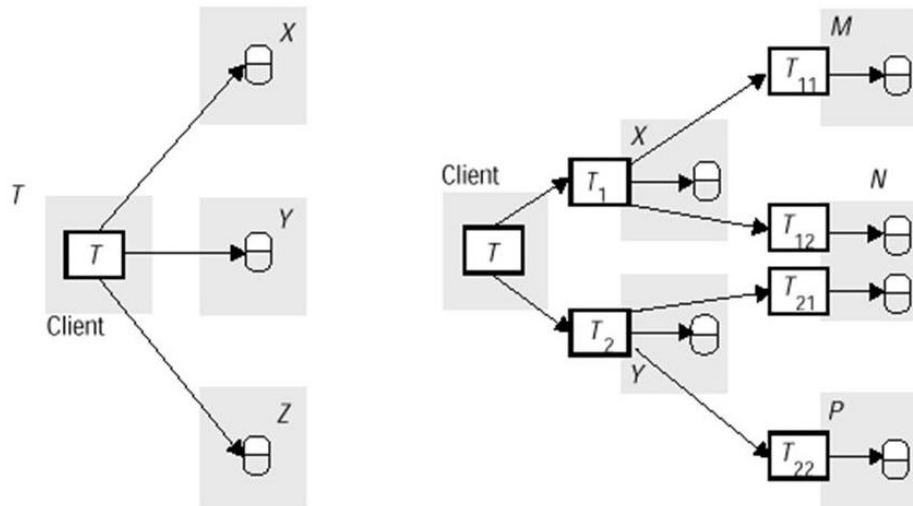
#### 5. Action Model

- A specialized model for **long-lived or cooperative transactions**.
- Transactions are broken into **actions** (small, recoverable units).
- Supports **undo/redo** via compensating actions.
- **Flexible execution and recovery**.
- **Use Case:** Business processes, collaborative work, or distributed workflows (e.g., e-commerce, travel booking).

## B. Based on Execution Scope and Structure

### 1. Flat Transaction

- A basic, single-level transaction that runs from start to finish without internal sub-transactions.
- All operations are **executed sequentially**.
- It either **commits fully** or **rolls back** completely (atomic).
- **Simplest** type of transaction.
- **Example:** A banking system where money is transferred from Account A to Account B — debit and credit happen in the same flat transaction.



Flat and nested transaction

### 2. Nested Transaction

- A transaction that contains **sub-transactions**, forming a hierarchy.
- Parent transaction spawns **child transactions**.
- Sub-transactions can **commit independently**, but final commit of the parent determines overall success.
- Supports **parallelism and modularity**.
- **Example:** An e-commerce order process may have sub-transactions for:
  - Checking inventory
  - Processing payment
  - Arranging shipping
  - Each of these can be treated as sub-transactions of the main order transaction.

### 3. Distributed Transaction

- **Definition:** A transaction that accesses **multiple databases or sites** in a distributed system.
- Operates over a **network of databases**.
- Requires coordination (e.g., **2-phase commit protocol**) to ensure atomicity across sites.
- Can be **flat** or **nested**.
- **Example:** A mobile wallet transaction updates balances on servers located in two different cities/databases.

### 4. Global Transaction

- A transaction **initiated by a global application** and may include multiple **local transactions** on different sites.
- Managed by a **global transaction manager**.
- Ensures **global consistency** across participating systems.
- Typically composed of multiple distributed transactions.

- **Example:** A cloud-based ERP system executing a company-wide monthly financial closure across all departments.

## 5. Local Transaction

- A transaction that executes **only on one site/database**.
- No interaction with other sites.
- Simpler to execute and manage.
- Does not require distributed coordination protocols.
- **Example:** A university student updating their course registration in a single university database.

| Transaction Type | Scope                    | Hierarchy          | Complexity | Use Case                            |
|------------------|--------------------------|--------------------|------------|-------------------------------------|
| Flat             | Single site or multiple  | No sub-tasks       | Low        | Simple tasks like bank transfer     |
| Nested           | Single or multiple sites | Has sub-tasks      | Medium     | Modular tasks like order processing |
| Distributed      | Multiple sites/databases | May be flat/nested | High       | Multi-location ATM transaction      |
| Global           | Multiple sites/databases | Oversees locals    | High       | Enterprise-level apps like SAP      |
| Local            | One database/site only   | Flat               | Low        | Simple local DB operations          |

# CH4: RELIABILITY ISSUES IN DDBSS

## Contents

|                                                |   |
|------------------------------------------------|---|
| Reliability in DDBMS .....                     | 1 |
| Failures in DDBMS.....                         | 1 |
| Techniques to Ensure Reliability in DDBMS..... | 2 |
| Commit Protocols .....                         | 3 |
| 1. Two-Phase Commit Protocol (2PC) .....       | 3 |
| Coordinator Timeout Scenarios .....            | 5 |
| Participant Timeout Scenarios .....            | 5 |
| 2. Three-Phase Commit Protocol (3PC).....      | 6 |
| Coordinator Timeouts .....                     | 7 |
| Participant Timeouts .....                     | 7 |

## Reliability in DDBMS

Reliability in a DDBS refers to the ability of the system to continue functioning correctly and provide consistent and correct data access even in the presence of various failures such as hardware malfunctions, communication errors, or software crashes.

- Since a DDBMS operates across multiple, networked sites, it is more vulnerable to partial failures.
- Hence, ensuring reliability is critical tekeko maintain data consistency, availability, and fault tolerance.

## Failures in DDBMS

Failures in DDBMS can occur due to many reasons. These failures disrupt normal operations and can compromise data reliability if not handled properly.

### a. Site Failure

- A complete site (machine or node) becomes non-operational.
- All the databases, transactions, and services running on that site become unavailable.
- Causes: Power failure, hardware crash, system crash.

### b. Link Failure

- The communication link between two or more sites is broken or disrupted.
- Affects the ability of sites to coordinate and access distributed data.
- Causes: Network cable disconnection, switch/router failure, poor connectivity.

### c. Transaction Failure

- An individual transaction fails due to logical errors or deadlocks.
- Only the specific transaction is affected, not the entire site.
- Causes: Programming bugs, constraint violations, deadlock, timeout.

#### **d. Media Failure**

- A storage device (like hard disk or SSD) fails, leading to loss or corruption of data.
- Affects the physical storage of data at a site.
- Causes: Hardware malfunction, disk crash, file system corruption.

#### **e. Communication Failure**

- Interruptions in message transfer between different sites or components.
- Causes inconsistent or lost transaction messages.
- Causes: Network congestion, protocol errors, or failures in communication software.

#### **f. Coordinator Failure**

- In a distributed transaction, if the site coordinating (e.g., in 2-phase commit) fails, the whole transaction may hang or become inconsistent.
- Very critical in distributed systems

## Techniques to Ensure Reliability in DDBMS

#### **a. Redundancy and Replication**

- Maintain multiple copies of data (replicas) across different sites.
- If one site fails, data can be retrieved from another.
- **Types:**
  - Full Replication: All data at all sites.
  - Partial Replication: Only critical data replicated.

#### **b. Distributed Commit Protocols**

- Ensure that all sites either commit or abort a distributed transaction together.
- **Two-Phase Commit (2PC):**
  - Phase 1: Prepare – All participants agree to commit.
  - Phase 2: Commit – If all agreed, commit; else, abort.
- **Three-Phase Commit (3PC):**
  - Adds an extra phase to avoid blocking in case of coordinator failure.

#### **c. Logging and Checkpointing**

- Logs record all actions taken by transactions to allow recovery.
- Checkpoints store the state of the system periodically to minimize recovery time.
- Used for rollback and redo during recovery.

#### **d. Failure Detection and Recovery**

- Use heartbeat signals or timeouts to detect failed sites or links.
- Automatic recovery procedures restart services or reroute queries to available replicas.

#### **e. Backup and Restore**

- Regular **backups** ensure that data can be restored in case of permanent media failure.
- Useful for long-term protection and disaster recovery.

#### **f. Quorum-based Protocols**

- Ensure consistency in replicated data.
- Read and write operations must be acknowledged by a **quorum (majority) of replicas**.
- Helps in fault tolerance and preventing inconsistency.

## g. Distributed Concurrency Control

- Use protocols like **Distributed Locking**, **Timestamp Ordering**, or **Optimistic Concurrency Control**.
- Prevents conflicts and ensures correctness even in the presence of concurrent transactions on different sites.

## Commit Protocols

In a **Distributed Database**, a single transaction may involve **multiple sites or databases**. To ensure **atomicity** (i.e., all-or-nothing execution of the transaction), **commit protocols** are used to **coordinate the decision** (commit or abort) across all involved sites.

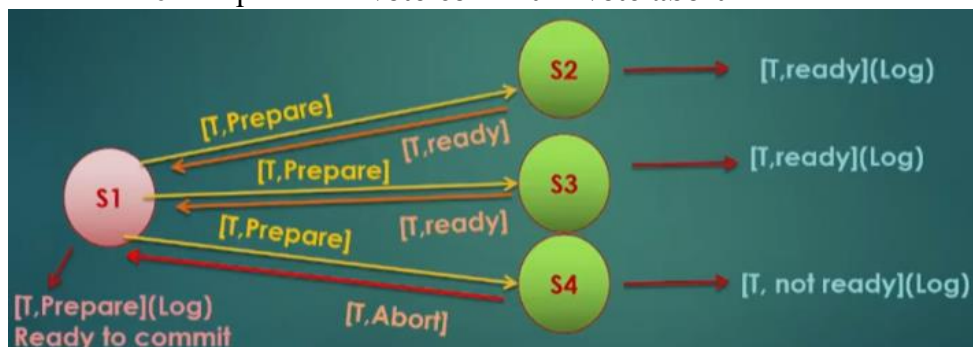
### 1. Two-Phase Commit Protocol (2PC)

- In distributed transaction, all its global and sub transaction should follow the property of Atomicity (all or none) to maintain consistency.
- 2 phase commit protocol ensures atomic commitment of distributed transactions.
- It checks if all site involved in the execution of the transaction are ready to commit or not.
- There are 2 modules here
  - **Coordinator**: the process at the site where the transaction originated and which controls execution
  - **Participants**: processes at the other site that participate in executing the transaction

#### ◆ Phases of 2PC:

##### □ Phase 1: Voting Phase (Prepare Phase)

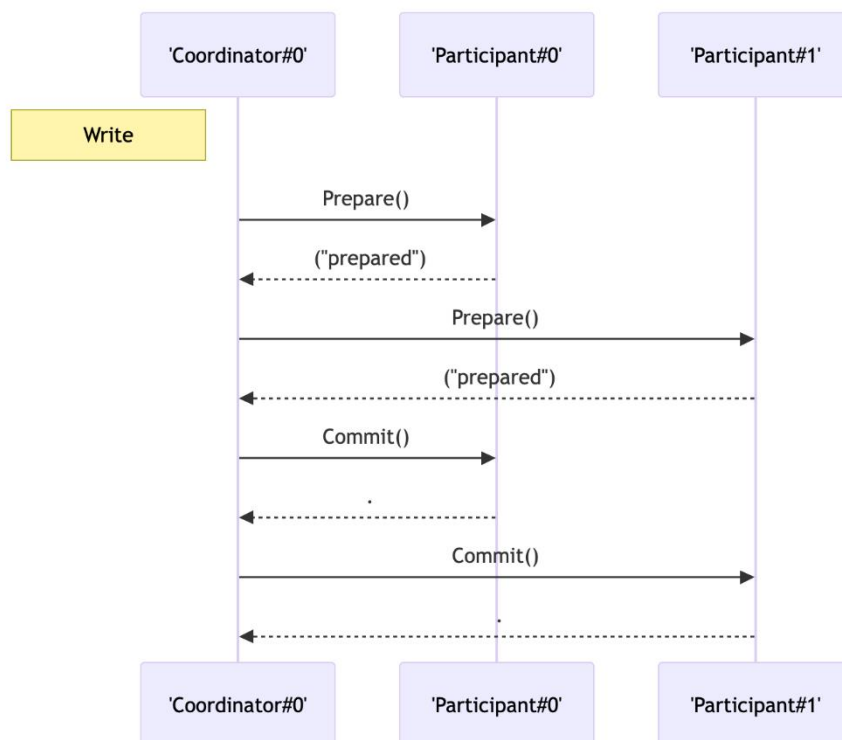
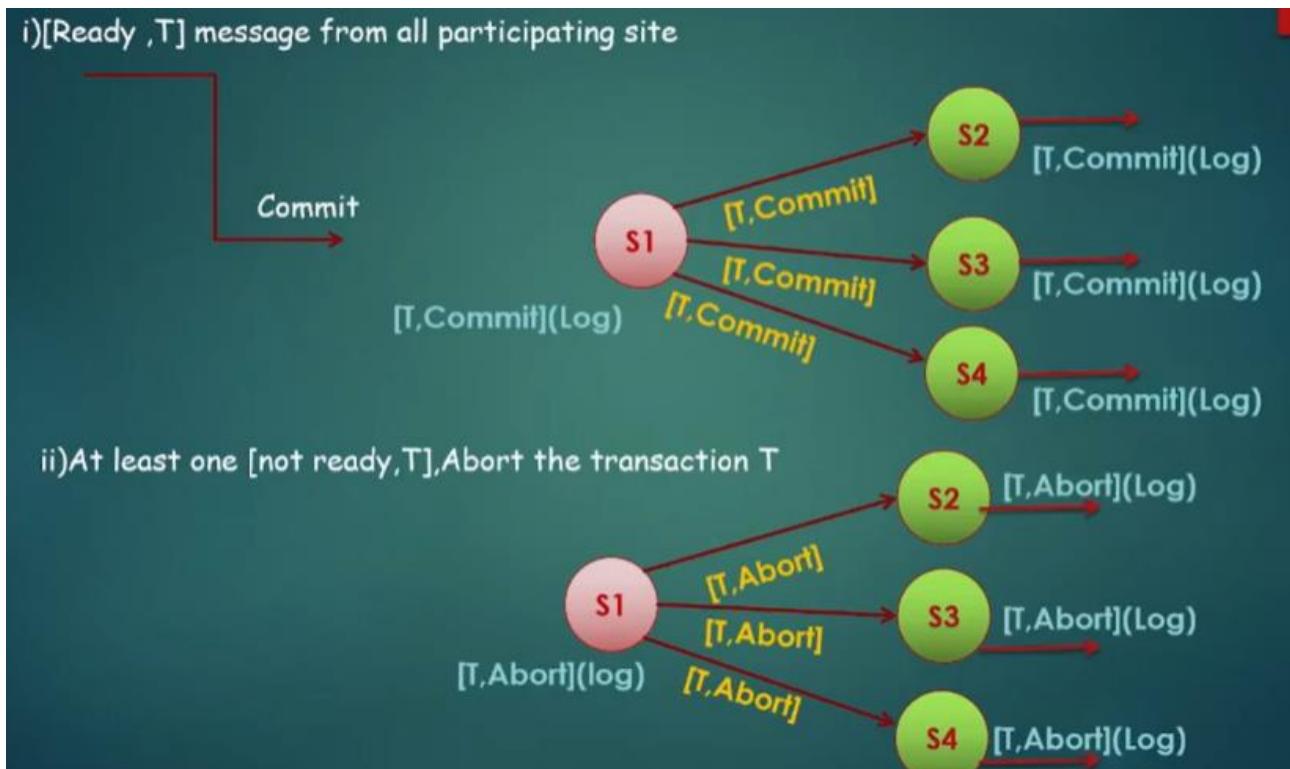
- **Coordinator** sends a *prepare* request to all participating sites (participants).
- Each participant does:
  - Writes the transaction details to its **log**.
  - Executes the transaction **up to the commit point**.
  - Replies with **vote-commit** or **vote-abort**.



As S4 is not ready, transaction would be aborted

##### □ Phase 2: Decision (Commit/Abort) Phase

- If **all participants vote to commit**, the coordinator sends a **global commit** message.
- If **any participant votes to abort**, the coordinator sends a **global abort** message.
- Participants then:
  - Commit or rollback the transaction.
  - Acknowledge the final outcome to the coordinator.



◆ **Advantages:**

- Simple and widely used.
- Ensures atomicity.

◆ **Disadvantages:**

- **Blocking Protocol:** If the coordinator crashes after sending the vote request but before sending the final decision, participants may be left **in doubt** and cannot proceed (i.e., they are **blocked**).
- Relies heavily on the coordinator's availability.

## Coordinator Timeout Scenarios

These happen **at the coordinator's side**, usually due to a delay or failure in communication with participants.

### Case 1: Coordinator Timeout While Waiting for Votes

- **Situation:** Coordinator sends Prepare request, but one or more participants **don't respond** in time.
- **Action:**
  - Coordinator assumes **those participants have failed**.
  - Aborts the transaction.
  - Sends **Global Abort** to the participants who responded.

### Case 2: Coordinator Timeout After Sending Global Decision

- **Situation:** Coordinator sends Global Commit or Global Abort, but doesn't receive acknowledgments from all participants.
- **Action:**
  - Coordinator can log the decision and retry sending the decision later.
  - The decision is **already logged**, so failure recovery is safe.

## Participant Timeout Scenarios

These are more critical — when participants are **waiting indefinitely**, it can cause **blocking**.

### Case 1: Participant Timeout Waiting for Prepare

- **Situation:** A participant expects a Prepare request from the coordinator but doesn't receive it.
- **Action:**
  - Do nothing (passive timeout).
  - Eventually the coordinator may retry or declare abort.
  - No risk here because the participant has not done anything yet.

### Case 2: Participant Timeout After Voting "Yes" (Waiting for Global Decision)

- **Situation:** Participant sends Vote-Commit but coordinator crashes before sending the decision.
- **Action:**
  - Participant is now in the **"uncertain state"** (this is the **blocking issue** of 2PC).
  - Participant **cannot decide** on its own whether to commit or abort.
  - Participant can:
    - **Wait** (until coordinator recovers),
    - **Query other participants** for the decision (if available), or
    - **Initiate termination protocol** (used in some systems).

### Case 3: Participant Timeout After Sending Acknowledgment

- **Situation:** Participant sent acknowledgment after commit/abort, but doesn't hear further.
- **Action:**
  - Safe to **assume transaction is complete**.
  - Clean up local logs and resources.

## Key Terms to Remember

- **Uncertain State:** When a participant has voted "Yes" and is waiting for a decision — it cannot act on its own.
- **Blocking Problem:** Occurs because participants in uncertain state cannot safely proceed if the coordinator fails.

## 2. Three-Phase Commit Protocol (3PC)

- **Blocking problem:** A major disadvantage of 2 PC protocol is faced when coordinator site fails and this results in blocking problem, as the participants are uncertain whether to commit or abort.
- **3PC is an extension of 2PC:** that avoids blocking even when the coordinator fails

### ◆ Phases of 3PC:

#### □ Phase 1: CanCommit Phase

- Coordinator sends a **CanCommit?** request (like "Are you ready?") to all participants.
- Participants respond with **Yes** or **No**.

#### □ Phase 2: PreCommit Phase

- If all respond **Yes**, the coordinator sends a **PreCommit** message.
- Participants:
  - Record a "pre-commit" log entry.
  - Prepare to commit, but do **not** actually commit yet.
  - Send an **acknowledgment** to the coordinator.

#### □ Phase 3: DoCommit Phase

- Once all participants have acknowledged PreCommit, the coordinator sends the **DoCommit** message.
- All participants **commit** the transaction and confirm the commit.

### How Blocking problem is handled?

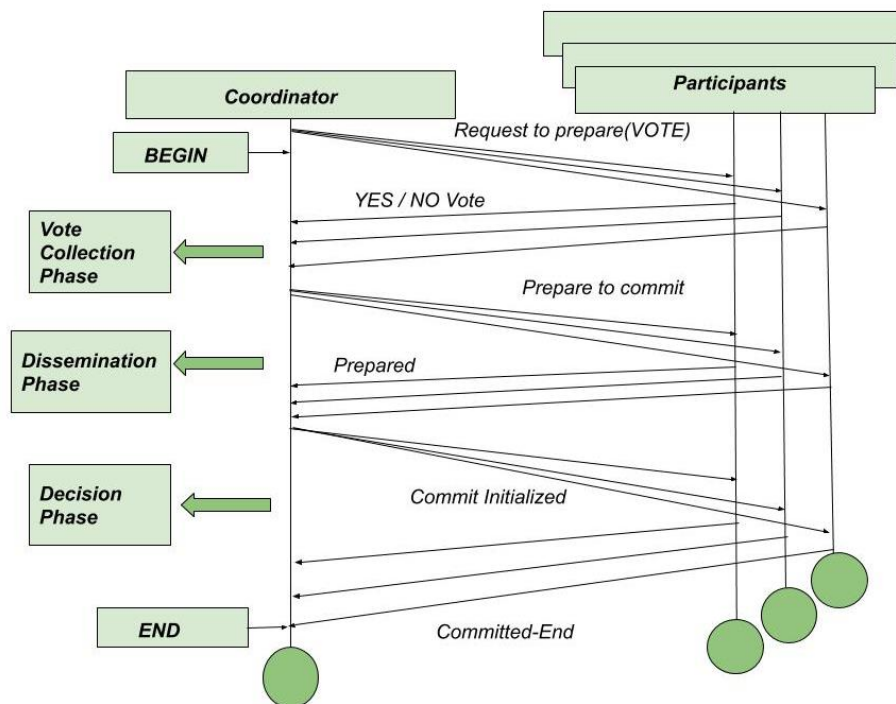
- If the coordinator fails, new coordinator is chosen which communicates with the remaining sites to check if old coordinator site had decided to commit.
- If a failure occurs, participants can use logs to decide whether to commit or abort independently.

### ◆ Advantages:

- **Non-blocking:** If a failure occurs, participants can use logs to decide whether to commit or abort independently.
- Better fault tolerance than 2PC.

### ◆ Disadvantages:

- **More overhead:** Involves an extra phase and more messages.
- Slower than 2PC in normal conditions.



## Coordinator Timeouts

- ◆ **Case 1: Timeout Waiting for "Yes"/"No" from Participants (CanCommit Phase)**
  - **Action:** Coordinator aborts the transaction and sends Global Abort to responding participants.
- ◆ **Case 2: Timeout Waiting for Acks after PreCommit**
  - **Action:** Coordinator may resend PreCommit; if still no response, assume participant has failed and proceed based on logs.
- ◆ **Case 3: Timeout Waiting for Acks after DoCommit**
  - **Action:** Coordinator considers transaction committed; logs are updated. Retries sending DoCommit if needed.

## Participant Timeouts

- ◆ **Case 1: Timeout Waiting for CanCommit Request**
  - **Action:** Do nothing or abort locally if no info is available.
- ◆ **Case 2: Timeout After Saying "Yes", Waiting for PreCommit**
  - **Action:** Participant assumes coordinator failed before PreCommit — **safe to abort** (since 3PC ensures no commit decision was made yet).
- ◆ **Case 3: Timeout After PreCommit, Waiting for DoCommit**
  - **Action:** Participant is in **pre-commit state** (a safe state).
    - Can communicate with other participants.
    - If majority says "PreCommit" → **commit**.
    - If others say "Abort" or "No Info" → **abort**.

| Feature          | Two-Phase Commit (2PC)        | Three-Phase Commit (3PC)                   |
|------------------|-------------------------------|--------------------------------------------|
| Phases           | 2                             | 3                                          |
| Blocking         | Yes                           | No                                         |
| Failure Recovery | Coordinator failure can block | Can proceed using logs                     |
| Message Overhead | Fewer messages                | More messages                              |
| Use Case         | Simpler systems               | Critical systems needing high availability |

# CH5: PARALLEL DATABASE SYSTEMS

## Contents

|                                                |   |
|------------------------------------------------|---|
| Parallel System Architectures in DDBMS.....    | 1 |
| Goals of parallel database system .....        | 1 |
| Parameters for parallel database .....         | 1 |
| 1.Shared Memory Architecture .....             | 3 |
| 2. Shared Disk Architecture .....              | 3 |
| 3. Shared Nothing Architecture .....           | 3 |
| 4. Hierarchical Architecture.....              | 4 |
| 5. NUMA (Non-Uniform Memory Access).....       | 5 |
| 6. COMA (Cache-Only Memory Architecture) ..... | 5 |

## Parallel System Architectures in DDBMS

- Sometimes centralized systems are not much efficient to handle data with high data transfer rate, due to which we use parallel database systems.
- Parallel system architecture refers to how **processors**, **memory**, and **storage** components are organized and connected to **achieve high performance, reliability**, and **scalability** in distributed or parallel database environments.
- These architectures define how **multiple processors** coordinate to process transactions or queries over **distributed data** in intercommunication network.

## Goals of parallel database system

- **Improved Performance** – Execute queries and transactions faster using multiple processors.
- **High Throughput** – Support many queries and transactions simultaneously.
- **Scalability** – Maintain performance as data volume or number of users grows.
- **Fault Tolerance** – Continue operating smoothly even if some components fail.
- **Load Balancing** – Distribute work evenly across all processing nodes.
- **Efficient Resource Utilization** – Maximize use of CPU, memory, and disk resources.
- **Data Availability** – Ensure continuous access to data across distributed sites.
- **Cost-Effectiveness** – Achieve high performance without needing expensive supercomputers.

## Parameters for parallel database

1. **Response Time:** Time taken to complete a single task (query or transaction).

$$\text{Response Time} = \text{Time from query start to finish}$$

◆ Goal: Lower response time = better performance.

◆ Example:

- Query on 1 node: 20 seconds
- Query on 4 parallel nodes: 6 seconds → Response time has **improved** due to parallelism.

2. **Speedup:** How much faster a parallel system performs a task compared to a single-processor system.

$$\text{Speedup}(S_p) = \frac{T_1}{T_p}$$

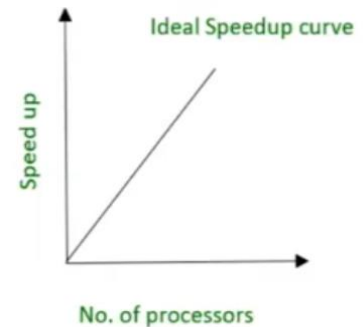
$T_1$  = time on 1 processor

$T_p$  = time on  $p$  processors

◆ Goal: Higher speedup = better parallel efficiency.

◆ Example:

- Time with 1 processor = 100 sec
- Time with 4 processors = 25 sec



3. **Linear Speedup:** When the speedup is proportional to the number of processors used.

◆ Ideal Condition:

$$S_p = p$$

•  $S_p$ : Speedup achieved

•  $p$ : Number of processors used.

◆ Example:

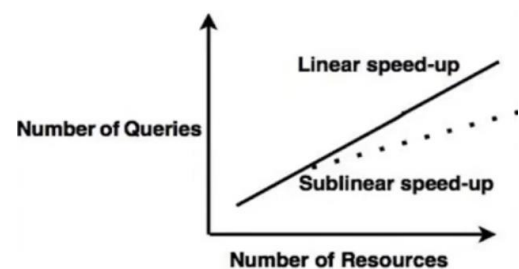
- 1 processor → 100 sec
- 2 processors → 50 sec
- 4 processors → 25 sec This shows linear speedup (doubling processors halves time).

4. **Sub-linear Speedup:** Speedup is less than the number of processors used (common in practice).

◆ Reason: Overheads like communication, synchronization, or load imbalance.

◆ Example:

- 1 processor → 100 sec
- 4 processors → 35 sec
- So  $S_4 = 100/35 = 2.86 < 4$  (which is in case of linear)



5. **Scaleup:** Measures a system's ability to handle a proportionally larger workload as more resources are added.

$$\text{Scaleup} = \frac{T_{1,W}}{T_{p,W'}}$$

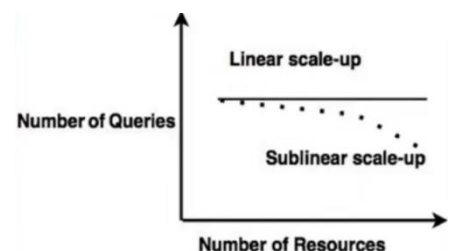
•  $T_{1,W}$  = time for workload  $W$  on 1 processor

•  $T_{p,W'}$  = time for  $p$  processors with workload  $W'$ , where  $W' = p \times W$

◆ Goal: Maintain same response time even if data and processors both scale.

◆ Example:

- 1 processor takes 1 hour for 10 GB
- 4 processors take 1 hour for 40 GB → Good **scaleup**



| Parameter          | Formula                    | Interpretation                                     |
|--------------------|----------------------------|----------------------------------------------------|
| Response Time      | Direct measurement         | Time to complete one task                          |
| Speedup            | $S_p = \frac{T_1}{T_p}$    | How much faster $p$ processors are vs 1            |
| Linear Speedup     | $S_p = p$                  | Ideal case                                         |
| Sub-linear Speedup | $S_p < p$                  | Real-world case due to overheads                   |
| Scaleup            | $\frac{T_{1,W}}{T_{p,W'}}$ | Efficiency as workload and resources both increase |

## 1. Shared Memory Architecture

Multiple processors share a single memory space and access it through a common bus or interconnect.

- All processors access **global shared memory**.
- Each processor may have its own **cache**.
- Often uses **mutexes or semaphores** for synchronization.

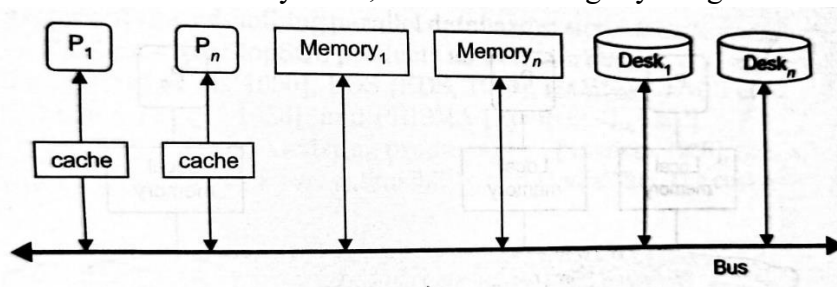
### ◆ Advantages:

- Easy to program.
- Fast communication between processors.
- Suitable for **tightly coupled systems**.

### ◆ Disadvantages:

- **Scalability limits** due to memory contention and bus bottlenecks.
- More complex to manage **cache coherence**.

◆ **Use Case:** Small to medium-sized systems; centralized or tightly integrated DDBMS.



## 2. Shared Disk Architecture

Each processor has its own private memory, but all processors share **common disk storage**.

- Processors communicate through **message passing** or interconnects.
- Disk contains shared data accessible to all nodes.
- Common in **high availability clusters** (like Oracle RAC).

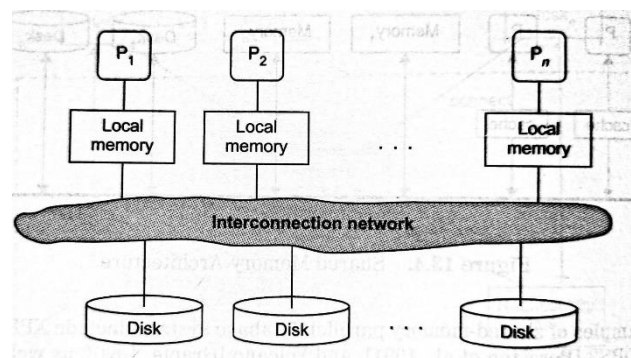
### ◆ Advantages:

- Easy data sharing.
- High fault tolerance – if one node fails, others can take over.

### ◆ Disadvantages:

- Disk contention and concurrency control overhead.
- Requires **complex coordination** (e.g., distributed locks).

◆ **Use Case:** Fault-tolerant systems with shared storage, like clustered databases.



## 3. Shared Nothing Architecture

Each node (processor) has its own **private memory and private disk**. No component is shared.

- Data is **partitioned** across nodes.
- Communication is done via **message passing**.

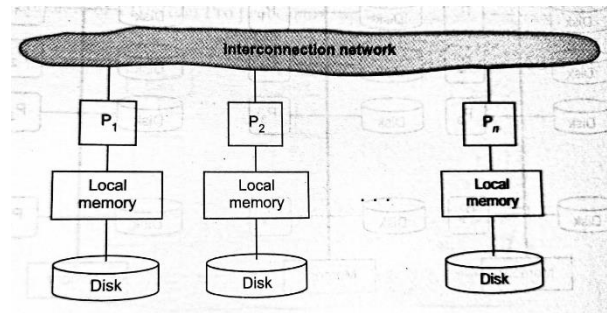
### ◆ Advantages:

- **Highly scalable** – each node works independently.

- No contention for memory or disk.
- Failure in one node doesn't affect others directly.

◆ **Disadvantages:**

- **Harder to manage** – requires careful partitioning and data distribution.
- Load balancing and fault tolerance are complex.



◆ **Use Case:** Large-scale distributed systems like Google Spanner, Amazon DynamoDB, NoSQL databases.

## 4. Hierarchical Architecture

Processors and resources are arranged in a **hierarchical tree-like structure**, often based on geography or role.

- Is a combination of shared memory and shared nothing architectures.
- The idea is to build a shared nothing model whose nodes are shared memory.
- Communication flows through levels of the hierarchy.

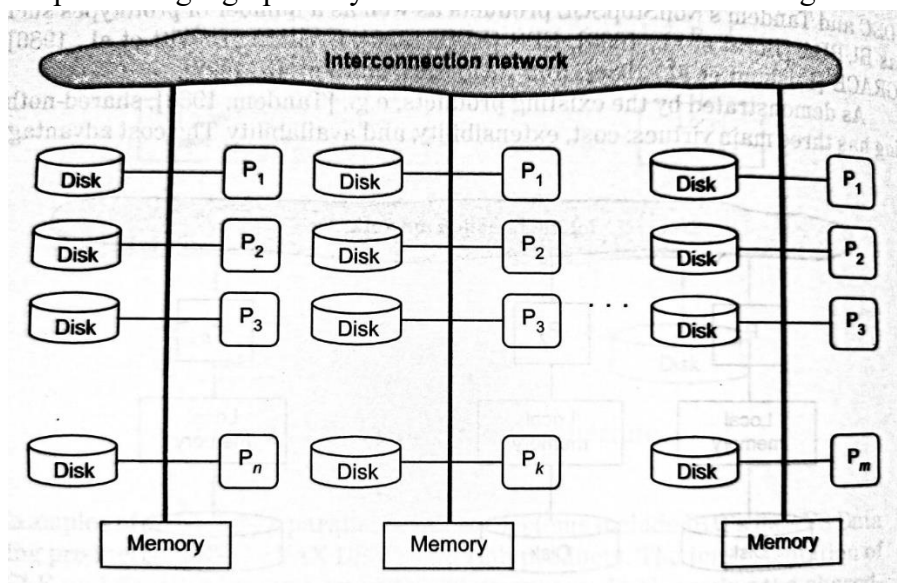
◆ **Advantages:**

- Flexibility and performance of shared memory.
- Secure like shared nothing.
- Easy to manage **large-scale systems**.
- Supports **data localization** and **regional autonomy**.

◆ **Disadvantages:**

- **Single points of failure** at higher levels.
- May introduce **latency** due to level-based communication.

◆ **Use Case:** Corporate or geographically distributed DDBMS with clear regional divisions.



## 5. NUMA (Non-Uniform Memory Access)

A memory architecture where **each processor has faster access to its local memory** and slower access to memory local to other processors.

- Combines features of shared and distributed memory.
- Maintains **global address space**.
- Memory access time **depends on location**.

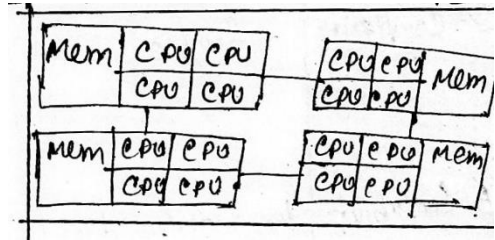
### ◆ Advantages:

- Good **performance scaling** with more processors.
- Avoids bottlenecks of purely shared memory.

### ◆ Disadvantages:

- **Programming complexity** increases.
- May suffer performance penalties if memory locality isn't optimized.

◆ **Use Case:** High-performance computing systems, hybrid architectures.



## 6. COMA (Cache-Only Memory Architecture)

A memory architecture where **all memory acts like a cache** — there is **no fixed home location** for data.

- Data moves dynamically between nodes.
- Cache coherence protocols maintain consistency.

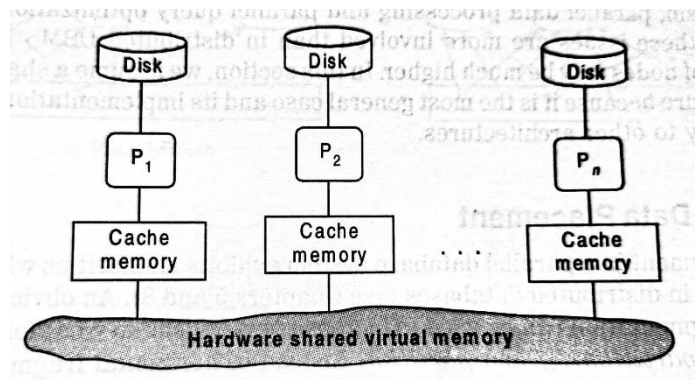
### ◆ Advantages:

- Efficient memory use — data automatically migrates where needed.
- High flexibility and **data locality**.

### ◆ Disadvantages:

- Very **complex hardware and coherence protocols**.
- Hard to implement at scale.

◆ **Use Case:** Experimental or specialized parallel systems in research or high-end computing.



| Architecture    | Shared Memory | Shared Disk | Shared Nothing | Hierarchical  | NUMA        | COMA        |
|-----------------|---------------|-------------|----------------|---------------|-------------|-------------|
| Memory          | Shared        | Private     | Private        | Mixed         | Distributed | Fully cache |
| Disk            | Shared        | Shared      | Private        | Mixed         | Mixed       | Distributed |
| Scalability     | Low-Medium    | Medium      | High           | High          | High        | Medium      |
| Fault Isolation | Low           | Medium      | High           | High          | Medium      | Medium      |
| Complexity      | Low           | Medium      | High           | Medium        | High        | Very High   |
| Typical Use     | Centralized   | Clusters    | Large DDBMS    | Global setups | HPC         | Research    |

# CH6: ADVANCED TOPICS

## Contents

|                                          |   |
|------------------------------------------|---|
| Mobile Databases .....                   | 1 |
| Distributed Object Management.....       | 2 |
| Multi-databases (MDBS) .....             | 2 |
| Types of MDBS.....                       | 3 |
| 1. Based on Autonomy .....               | 3 |
| 2. Based on Heterogeneity .....          | 3 |
| 3. Based on Distribution .....           | 4 |
| 4. Based on Data Model Integration ..... | 4 |
| 5. Based on System Architecture .....    | 5 |

## Mobile Databases

A Mobile Database is a type of distributed database system where clients or users move physically, and the network connection is not stable. The database must support intermittent connectivity, disconnections, and limited bandwidth, commonly in mobile computing environments (e.g., smartphones, tablets, vehicles).

### ◆ Architecture:

1. **Mobile Host (MH)** – A portable device that runs applications and stores a subset of data.
2. **Fixed Host (FH)** – Traditional stationary servers.
3. **Base Station (BS)** – Acts as a communication interface between mobile and fixed hosts.

### ◆ Challenges:

- **Frequent Disconnections** due to mobility
- **Limited Resources** (battery, storage, bandwidth)
- **Synchronization** issues between mobile and central databases
- **Security & Privacy** concerns in wireless communication

### ◆ Solutions and Features:

- **Caching:** Frequently used data is stored locally to reduce server access.
- **Replication:** Data may be partially or fully replicated on mobile devices.
- **Synchronization Techniques:** To sync data when reconnected (e.g., conflict resolution policies).
- **Lightweight Transactions:** Optimized for mobile resources.

# Distributed Object Management

Distributed Object Management (DOM) extends DDBMS concepts by using objects (from object-oriented programming) as the basic units of data instead of traditional records and tables.

DOM systems manage distributed object-oriented databases (DOODB) where data and behavior are encapsulated together. **Key Features:**

- **Encapsulation:** Data and operations (methods) are bundled into objects.
- **Object Identity:** Each object has a unique identity, persistent over time and across sites.
- **Inheritance:** Objects can inherit behavior from other classes.
- **Complex Objects:** Can store lists, sets, arrays — more than just simple data types.
- **Transparency:** Users do not need to know the location of objects.

## ◆ Challenges in Distributed Setup:

- **Object Migration:** Moving objects between sites without breaking references.
- **Object Replication:** Keeping multiple copies consistent.
- **Remote Method Invocation (RMI):** Calling object methods remotely.
- **Concurrency Control:** Handling simultaneous updates on object attributes.
- **Garbage Collection:** Managing deletion across distributed sites.

## ◆ Technologies Used:

- **CORBA (Common Object Request Broker Architecture)**
- **Java RMI**
- **Object Data Management Group (ODMG) standards**

# Multi-databases (MDBS)

A Multidatabase System (MDBS) is a system that provides a unified interface to access multiple autonomous databases that may be:

- Heterogeneous (different data models, query languages)
- Distributed over multiple locations
- Independently managed

**Goal:** Allow users to query and update **multiple databases** through a **single global schema/interface**, hiding the complexity of differences.

## ◆ Types of Heterogeneity:

1. **Data Model Heterogeneity** – Different databases (e.g., relational, NoSQL, object-oriented).
2. **Query Language Heterogeneity** – Different languages (SQL, XQuery, etc.)
3. **Semantic Heterogeneity** – Same term used differently across databases.
4. **Schema Heterogeneity** – Structural differences in schemas.

## ◆ MDBS Architecture:

1. **Local DBMSs** – Existing independent databases.
2. **Global Schema** – Unified representation of all databases.
3. **Middleware Layer** – Handles query translation, decomposition, and integration.

## ◆ Functions of MDBS:

- **Query Decomposition:** Breaking a global query into local subqueries.

- **Result Integration:** Merging results from different sources.
- **Schema Mapping:** Translating queries between global and local schemas.
- **Conflict Resolution:** Handling semantic and structural conflicts.

#### ◆ Advantages:

- **Unified Access:** Single interface for many databases.
- **Autonomy Preservation:** Local databases remain independent.
- **Scalability:** Easy to add/remove databases.

#### ◆ Challenges:

- Maintaining **global consistency**
- **Query optimization** across different DBMSs
- Handling **data security and access control** across organizations

## Types of MDBS

### 1. Based on Autonomy

✓ **Tightly Coupled MDBS:** All participating databases are integrated closely and controlled by a central authority.

- Centralized management.
- Less autonomy for individual databases.
- Often built from scratch or with similar systems.
- **Advantages:**
  - Easier consistency and control.
  - Better performance due to tighter integration.
- **Disadvantages:**
  - Difficult to integrate existing autonomous systems.

✓ **Loosely Coupled MDBS:** Participating databases are **independent and autonomous**, integrated using middleware or a federation system.

- Each database manages its own schema, data, and operations.
- Communication via a coordination layer.
- **Advantages:**
  - Supports heterogeneity and scalability.
  - Easier to plug in existing databases.
- **Disadvantages:**
  - Harder to enforce global constraints and consistency.

### 2. Based on Heterogeneity

✓ **Homogeneous MDBS:** All databases in the system use the same DBMS, data model, and query language. **Example:** Multiple MySQL servers integrated in a single system.

- **Advantages:**
  - Simplified integration and query processing.
  - Easier optimization and transaction management.
- **Disadvantages:**
  - Limited flexibility (can't integrate different types of systems).

✓ **Heterogeneous MDBS:** Databases use different DBMSs, data models (e.g., relational, object-oriented), and query languages (e.g., SQL, XQuery). **Example:** Combining Oracle (relational) and MongoDB (NoSQL).

- **Advantages:**
  - Greater flexibility to integrate diverse systems.
- **Disadvantages:**
  - Complex query translation and schema mapping.
  - More effort to ensure semantic consistency.

### 3. Based on Distribution

✓ **Federated Databases:** A type of loosely coupled MDBS where multiple databases are **linked together under a common framework** (federation).

- Each database retains autonomy.
- Users can access and query across all databases via a global interface.
- **Types:**
  - **Strong Federation:** Shared data dictionary and control.
  - **Weak Federation:** Minimal coordination, more autonomy.
- **Advantages:**
  - Supports local autonomy while providing global access.
- **Disadvantages:**
  - Complex query planning and transaction management.

✓ **Non-Federated MDBS:** A collection of databases with **no formal federation**, often connected via application-level integration.

- No unified schema or dictionary.
- May lack transparency.
- **Advantages:**
  - Quick integration of existing systems.
- **Disadvantages:**
  - Poor query support and coordination.
  - No standard interface.

### 4. Based on Data Model Integration

✓ **Global Schema MDBS:** A **single unified schema** is created to represent all the local databases in the system.

- Users write queries based on this global schema.
- The system translates global queries into local subqueries.
- **Advantages:**
  - Simple and intuitive for users.
  - Good data abstraction.
- **Disadvantages:**
  - Difficult to maintain consistency when local schemas change.
  - High complexity in schema design.

✓ **Schema Mapping-based MDBS:** Instead of creating a single global schema, the system uses **mappings** between local schemas to resolve heterogeneity.

- More dynamic and flexible.
- Queries are rewritten using mapping rules.
- **Advantages:**
  - Easier to maintain as systems evolve.
  - Suitable for loosely coupled, heterogeneous systems.
- **Disadvantages:**
  - Complex mapping and query rewriting logic.
  - May affect performance.

## 5. Based on System Architecture

✓ **Mediator-based MDBS:** Uses a **mediator layer** between the user and data sources to manage communication, integration, and translation.

- Mediator receives the user query, decomposes it, routes it to the right databases, and integrates the results.
- Transparent to users.
- **Advantages:**
  - Simplifies user interaction.
  - Good for heterogeneous environments.
- **Disadvantages:**
  - Performance overhead.
  - Single point of failure if not replicated.

✓ **Agent-based MDBS:** Uses **software agents** (autonomous programs) to represent and interact with individual databases.

- Each agent knows about its own local database.
- Agents cooperate to process user queries.
- **Advantages:**
  - Flexible and adaptive.
  - Suitable for dynamic and mobile environments.
- **Disadvantages:**
  - Coordination among agents can be complex.
  - Security and trust between agents is a concern.